

SOLIDWORKS API (.NET VBA C#) — Examples (17/50):

Create Swept Flange Using Gauge Table Ex- Create_Variable_Pitch_Helix_Example_VBNE

SOLIDWORKS API Documentation (converted from CHM)

Create Swept Flange using Gauge Table Example (VB.NET)

This example shows how to create a swept flange using a bend table on a non-sheet metal part.

```
'=====
```

```
'Preconditions:
```

```
'1. Ensure that the paths to templates and gauge tables are valid.
'2. Open an Immediate window.
'3. Press F5 repeatedly and inspect the Immediate window and FeatureManager design tree as instructed.
'
```

```
'Postconditions:
```

```
'1. Creates Sketch1 for the sweep path.
'2. Creates Sketch2 for the sweep profile.
'3. Creates Swept Flange1 using a gauge table installed with SOLIDWORKS.
'4. Displays gauge table parameters.
'5. Modifies Swept Flange1 to override the gauge number, bend radius, and gauge thickness.
'6. Displays new gauge parameters.
'
```

```
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swPart As PartDoc
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swFeat As Feature
```

```
        Dim swFeatMgr As FeatureManager
```

```
        Dim swSweptFlangeFeatureData As SweptFlangeFeatureData
```

```
        Dim featData As SweptFlangeFeatureData
```

```
        Dim smGaugeTableParam As SheetMetalGaugeTableParameters
```

```
        Dim skSegment As SketchSegment
```

```
        Dim myRefPlane As RefPlane
```

```
        Dim swFeatNameID As Integer
```

```
        Dim swSketch(0) As Feature
```

```
        Dim boolstatus As Boolean
```

```
        Dim errCode As Integer
```

```
        Dim swSheetWidth As Double
```

```
        Dim swSheetHeight As Double
```

```
        Dim gaugePath As String = Nothing
```

```
        Dim gaugeCount As Integer
```

```
        Dim gaugeNumbers As Object
```

```
        Dim i As Integer
```

```
        Dim radiiCount As Integer
```

```
        Dim radii As Object
```

```
        Dim newgaugePath As String = Nothing
```

```
        swSheetWidth = 0
```

```
        swSheetHeight = 0
```

```
        swModel = swApp.NewDocument("E:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\Tutorial\part.prt", 0,
swSheetWidth, swSheetHeight)
```

```
        swPart = swModel
```

```
        swModel = swApp.ActiveDoc
```

```
        skSegment = swModel.SketchManager.Create3PointArc(-0.058601, -0.015313, 0#, -0.003828, -0.021603, 0#, 0.002265, -0.05642,
0#)
```

```
        swModel.ClearSelection2(True)
```

```

swModel.SketchManager.InsertSketch(True) 'Sketch1
swModel.ClearSelection2(True)

0)
boolstatus = swModel.Extension.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", -0.003828, -0.021603, 0, True, 0, Nothing,
boolstatus = swModel.Extension.SelectByID2("Arc1@Sketch1", "EXTSKETCHSEGMENT", 0.00285760095397308, -0.0342380271903227, 0,
True, 1, Nothing, 0)

myRefPlane = swModel.FeatureManager.InsertRefPlane(4, 0, 2, 0, 0, 0)

boolstatus = swModel.Extension.SelectByID2("Plane1", "PLANE", 0.0103901931573406, -0.00917038599747196,
-0.00622490227027586, True, 0, Nothing, 0)

swModel.SketchManager.InsertSketch(True)
skSegment = swModel.SketchManager.CreateLine(0#, 0#, 0#, 0#, 0.018316, 0#)
skSegment = swModel.SketchManager.CreateLine(0#, 0.018316, 0#, 0.008362, 0.035435, 0#)
swModel.ClearSelection2(True)
swModel.SketchManager.InsertSketch(True) 'Sketch2

swFeatNameID = swFeatureNameID_e.swFmSweptFlange
swFeatMgr = swModel.FeatureManager
swSweptFlangeFeatureData = swFeatMgr.CreateDefinition(swFeatNameID)

swModel.ClearSelection2(True)

'Select the sweep path
boolstatus = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
swSketch(0) = swModel.SelectionManager.GetSelectedObject6(1, -1)

swSweptFlangeFeatureData.Path = swSketch

'Select the sweep profile
boolstatus = swModel.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
swSweptFlangeFeatureData.Profile = swModel.SelectionManager.GetSelectedObject6(1, -1)

errCode = swSweptFlangeFeatureData.GetErrorCodes
Debug.Print("Swept flange definition error code: " + CStr(errCode))

Stop 'Inspect the Immediate window for the swept flange definition error

swSweptFlangeFeatureData.UseMaterialSheetMetalParameters = False
swSweptFlangeFeatureData.UseGaugeTable = True

Debug.Print("Use gauge table? " + CStr(swSweptFlangeFeatureData.UseGaugeTable))

smGaugeTableParam = swSweptFlangeFeatureData.GetGaugeTableParameters
boolstatus = smGaugeTableParam.GetGaugeTablePath(gaugePath)

If boolstatus = False Then
    smGaugeTableParam.SetGaugeTablePath("E:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\LANG\ENGLISH\SHEET METAL GAUGE
TABLES\BEND ALLOWANCE MM SAMPLE.XLS")
End If

boolstatus = smGaugeTableParam.GetGaugeTablePath(gaugePath)
Debug.Print("Got gauge table path? " + CStr(boolstatus))
Debug.Print("Gauge table path: " + gaugePath)
Debug.Print("Process: " + smGaugeTableParam.Process)

gaugeCount = smGaugeTableParam.GetGaugeNumberCount()
Debug.Print("Gauge number count: " + CStr(gaugeCount))

Debug.Print("Available gauge numbers: ")

gaugeNumbers = smGaugeTableParam.GetAvailableGaugeNumbers()

```

```

For i = 0 To gaugeCount - 1
    Debug.Print(gaugeNumbers(i))
Next
Debug.Print("Current gauge number: " + smGaugeTableParam.GetCurrentGaugeNumber())

Stop 'Inspect the Immediate window for the current gauge number

radiiCount = smGaugeTableParam.GetRadiiCount()
Debug.Print("Bend radii count: " + CStr(smGaugeTableParam.GetRadiiCount()))

Debug.Print("Available bend radii: ")
radii = smGaugeTableParam.GetAvailableRadii()
For i = 0 To radiiCount - 1
    Debug.Print(radii(i))
Next
Debug.Print("Current bend radius: " + CStr(smGaugeTableParam.GetCurrentRadius))
Debug.Print("Thickness: " + CStr(smGaugeTableParam.GetThickness))
Debug.Print("Override thickness? " + CStr(smGaugeTableParam.OverrideThickness))
Debug.Print("Override bend radius? " + CStr(smGaugeTableParam.OverrideRadius))
Debug.Print("Reverse direction? " + CStr(smGaugeTableParam.ReverseDirection))

Stop 'Inspect the Immediate window for current bend radius and current gauge thickness

swSweptFlangeFeatureData.SetGaugeTableParameters(smGaugeTableParam)

swFeat = swFeatMgr.CreateFeature(swSweptFlangeFeatureData)

errCode = swSweptFlangeFeatureData.GetErrorCodes()
Debug.Print("Swept flange creation error code: " + CStr(errCode))
swModel.ClearSelection2(True)

Stop 'Inspect the Immediate window for the swept flange creation status
'Observe Sheet-Metal, Swept Flange1, and Flat-Pattern in the FeatureManager design tree

'Set new gauge number and override gauge thickness and bend radius
featData = swFeat.GetDefinition()
smGaugeTableParam = featData.GetGaugeTableParameters

smGaugeTableParam.ReverseDirection = True
smGaugeTableParam.SetCurrentGaugeNumber("Gauge 3")
smGaugeTableParam.SetThickness(0.006, True)
smGaugeTableParam.SetRadius(0.006, True)

featData.SetGaugeTableParameters(smGaugeTableParam)
boolstatus = swFeat.ModifyDefinition(featData, swModel, Nothing)
Debug.Print("Swept flange modification status: " + CStr(boolstatus))

Stop 'Inspect the Immediate window for the swept flange modification status

'Get new gauge number
boolstatus = featData.AccessSelections(swModel, Nothing)
smGaugeTableParam = featData.GetGaugeTableParameters

boolstatus = smGaugeTableParam.GetGaugeTablePath(newgaugePath)
Debug.Print("Got gauge table path? " + CStr(boolstatus))
Debug.Print("Gauge table path: " + newgaugePath)

Debug.Print("Process: " + smGaugeTableParam.Process)

gaugeCount = smGaugeTableParam.GetGaugeNumberCount()
Debug.Print("Gauge number count: " + CStr(gaugeCount))

```

```

Debug.Print("Available gauge numbers: ")
gaugeNumbers = smGaugeTableParam.GetAvailableGaugeNumbers()

For i = 0 To gaugeCount - 1
    Debug.Print(gaugeNumbers(i))
Next
Debug.Print("Current gauge number: " + smGaugeTableParam.GetCurrentGaugeNumber())

Stop 'Inspect the Immediate window for the new gauge number

'Get new bend radius and gauge thickness
radiiCount = smGaugeTableParam.GetRadiiCount()
Debug.Print("Bend radii count: " + CStr(radiiCount))

Debug.Print("Available bend radii: ")
radii = smGaugeTableParam.GetAvailableRadii()
For i = 0 To radiiCount - 1
    Debug.Print(radii(i))
Next
Debug.Print("Current bend radius: " + CStr(smGaugeTableParam.GetCurrentRadius))

Debug.Print("Thickness: " + CStr(smGaugeTableParam.GetThickness))
Debug.Print("Override thickness? " + CStr(smGaugeTableParam.OverrideThickness))
Debug.Print("Override bend radius? " + CStr(smGaugeTableParam.OverrideRadius))
Debug.Print("Reverse direction? " + CStr(smGaugeTableParam.ReverseDirection))

featData.ReleaseSelectionAccess

Stop 'Inspect the Immediate window for the new bend radius and gauge thickness

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create and Edit Symmetric Mate Example (C#)

This example shows how to create and edit a symmetric mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo3.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a symmetric mate.
// 2. Press F5 to continue.
// 3. Changes the symmetry plane and the entities to mate.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CreateSymmetricMate_CSharp
{
    partial class SolidWorksMacro
    {

        private ModelDoc2 swModel;
        private bool boolstatus;
        private AssemblyDoc AsmDoc;
        private MateFeatureData MateData;
        private SymmetricMateFeatureData SymmMateData;
        private SelectionMgr selman;
        private object FaceVar;
        private Feature Feat;
        private object Plane;
        private Face2[] FaceArr = new Face2[2];
        public void Main()
        {

            swModel = (ModelDoc2)swApp.ActiveDoc;
            AsmDoc = (AssemblyDoc)swModel;

            MateData = (MateFeatureData)AsmDoc.CreateMateData(8); //Symmetric mate
            selman = (SelectionMgr)swModel.SelectionManager;

            SymmMateData = (SymmetricMateFeatureData)MateData;

            // Set the symmetry plane
            Plane = AsmDoc.FeatureByName("Front Plane");
            SymmMateData.SymmetryPlane = Plane;

            // Select the faces
            boolstatus = swModel.Extension.SelectByRay(-0.119833719900839, 0.14832954277739, -0.0138999787131979, -0.638789958006775, -0.000000000000000);
            boolstatus = swModel.Extension.SelectByRay(0.00598755999561718, 0.0437101841503704, -0.0138999787133685, -0.400036026775, -0.000000000000000);

            // Set the entities to mate
            FaceArr[0] = (Face2)selman.GetSelectedObject6(1, -1);
            FaceArr[1] = (Face2)selman.GetSelectedObject6(2, -1);
            FaceVar = FaceArr;
            SymmMateData.EntitiesToMate = FaceVar;
            SymmMateData.MateAlignment = 1;

            Feat = (Feature)AsmDoc.CreateMate(MateData);

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();

            Feat = swModel.Extension.GetLastFeatureAdded();
            Debug.Print("Feature type of mate created is " + Feat.GetTypeName2());

            MateData = (MateFeatureData)Feat.GetDefinition();

            swModel.ClearSelection2(true);

            SymmMateData = (SymmetricMateFeatureData)MateData;

            Plane = AsmDoc.FeatureByName("Top Plane");
            SymmMateData.SymmetryPlane = Plane;

            boolstatus = swModel.Extension.SelectByRay(-0.1227406900662738, 0.149346213190292, -0.0800800212867898, -0.64029473479625, -0.000000000000000);
            boolstatus = swModel.Extension.SelectByRay(0.00598755999561718, 0.0437101841503704, -0.0138999787133685, -0.400036026775, -0.000000000000000);
            FaceArr[0] = (Face2)selman.GetSelectedObject6(1, -1);
            FaceArr[1] = (Face2)selman.GetSelectedObject6(2, -1);
```

```
        FaceVar = FaceArr;

        SymmMateData.EntitiesToMate = FaceVar;

        Debug.Print("Symmetric mate alignment is " + SymmMateData.MateAlignment);

        Feat.ModifyDefinition(SymmMateData, swModel, null);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

}
```

Create and Edit Symmetric Mate Example (VBA)

This example shows how to create and edit a symmetric mate.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo3.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a symmetric mate.
' 2. Press F5 to continue.
' 3. Changes the symmetry plane and the entities to mate.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim face As SldWorks.Face2
Dim AsmDoc As SldWorks.AssemblyDoc
Dim MateData As SldWorks.MateFeatureData
Dim SymmMateData As SldWorks.SymmetricMateFeatureData
Dim selman As SelectionMgr
Dim FaceVar As Variant
Dim Feat As SldWorks.Feature
Dim Plane As Object
Dim EntToMate As Variant
Dim FaceArr(1) As SldWorks.Face2

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set AsmDoc = swModel

    Set MateData = AsmDoc.CreateMateData(8) 'Symmetric mate
    Set selman = swModel.SelectionManager

    ' Set the symmetry plane
    Set Plane = AsmDoc.FeatureByName("Front Plane")
    MateData.SymmetryPlane = Plane

    ' Select the faces
    boolstatus = swModel.Extension.SelectByRay(-0.119833719900839, 0.14832954277739, -1.38999787131979E-02, -0.638789958006775,
-0.241329918931549, -0.730552708418903, 1.94730543661126E-03, 2, False, 0, 0)
    boolstatus = swModel.Extension.SelectByRay(5.98755999561718E-03, 4.37101841503704E-02, -1.38999787133685E-02,
-0.400036026779312, -0.515038074910024, -0.758094294050284, 9.54271027477843E-04, 2, True, 0, 0)

    ' Set the entities to mate
    Set FaceArr(0) = selman.GetSelectedObject6(1, -1)
    Set FaceArr(1) = selman.GetSelectedObject6(2, -1)

    FaceVar = FaceArr
    MateData.EntitiesToMate = FaceVar

    MateData.MateAlignment = 1
    Set Feat = AsmDoc.CreateMate(MateData)

    swModel.GraphicsRedraw2

    Stop

    Set Feat = swModel.Extension.GetLastFeatureAdded
    Debug.Print "Feature type of mate created is " & Feat.GetTypeName2

    Set MateData = Feat.GetDefinition

    swModel.ClearSelection2 (True)

    Set SymmMateData = MateData

    Set Plane = AsmDoc.FeatureByName("Top Plane")
    SymmMateData.SymmetryPlane = Plane

    boolstatus = swModel.Extension.SelectByRay(-0.122740690662738, 0.149346213190292, -8.00800212867898E-02, -0.640294734796254,
5.58893693642409E-02, 0.766093356572332, 2.43053632782351E-03, 2, False, 0, 0)
    boolstatus = swModel.Extension.SelectByRay(5.98755999561718E-03, 4.37101841503704E-02, -1.38999787133685E-02,
-0.400036026779312, -0.515038074910024, -0.758094294050284, 9.54271027477843E-04, 2, True, 0, 0)

```

```
Set FaceArr(0) = selman.GetSelectedObject6(1, -1)
Set FaceArr(1) = selman.GetSelectedObject6(2, -1)

FaceVar = FaceArr

SymmMateData.EntitiesToMate = FaceVar

Debug.Print "Symmetric mate alignment is " & SymmMateData.MateAlignment

Call Feat.ModifyDefinition(SymmMateData, swModel, Nothing)

End Sub
```

Create Tab and Slot Example (C#)

This example shows how to create a tab and slot feature.

```
// -----
// Preconditions: Open public_documents\samples\whatsnew\sheet metal\tab_and_slot.sldprt.
//
// Postconditions:
// 1. Select Tab and Slot-Tab1 in the FeatureManager design tree;
//    5 tabs/slots created
// 2. Press F5 to continue.
// 3. Select Tab and Slot-Tab1 in the FeatureManager design tree;
//    1 tab/slot remains
//
// NOTE: Because the model is used elsewhere, do not save any changes to it.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateTabAndSlot_CSharp
{
    partial class SolidWorksMacro
    {
        private TabAndSlotFeatureData featdata;
        private TabAndSlotGroupData grp;
        private ModelDoc2 Part;
        private Edge swedge;
        private Face2 swface;
        private Feature feat;
        private TabAndSlotFeatureData newFeatData;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByRay(-0.0581596588411344, 0.0247654446213232, 0.000234517259855238, -0.57738154519998;
            swedge = (Edge)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            Part.ClearSelection2(true);

            boolstatus = Part.Extension.SelectByRay(-0.0133680887414585, -0.00908584763345743, -0.00199999999999534, -0.821234743714;
            swface = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            Part.ClearSelection2(true);

            featdata = (TabAndSlotFeatureData)Part.FeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmTabAndSlot);

            grp = (TabAndSlotGroupData)featdata.SelectionAddNewGroup();
            grp.Offset = true;
            grp.SelectionTabEdge = swedge;
            grp.SelectionSlotFace = swface;
            Part.FeatureManager.CreateFeature(featdata);

            System.Diagnostics.Debugger.Break();

            // Select Tab and Slot-Tab1 in the FeatureManager design tree; 5 tabs/slots created on the part
            // Press F5 to continue

            feat = (Feature)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            newFeatData = (TabAndSlotFeatureData)feat.GetDefinition();
            TabAndSlotGroupData[] var = null;
            newFeatData.AccessSelections(Part, null);
            var = (TabAndSlotGroupData[])newFeatData.SelectionGetGroups();
            var[0].Offset = false;
            var[0].SpacingType = 0;
            var[0].SpacingNumberOfInstances = 1;
            feat.ModifyDefinition(newFeatData, Part, null);

            // Select Tab and Slot-Tab1 in the FeatureManager design tree; only one tab and slot remain

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Table Pattern Example (C#)

This example shows how to create a table pattern feature.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\tablepattern.sldprt.
// 2. Delete TPattern1.
//
// Postconditions:
// 1. Creates a new TPattern1.
// 2. Inspect the FeatureManager design tree and the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateTablePattern_CSharp
{
    partial class SolidWorksMacro
    {
        private TablePatternFeatureData swFeatData;
        private ModelDoc2 Part;
        private Feature swFeat;
        private FeatureManager swFeatMgr;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("Coordinate System1", "COORDSYS", 0, 0, 0, true, 16, null, 0);
            boolstatus = Part.Extension.SelectByID2("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, true, 4, null, 0);
            Part.ActivateSelectedFeature();

            double[] swPointArray = new double[6];
            swPointArray[0] = 0.04;
            swPointArray[1] = 0;
            swPointArray[2] = 0;
            swPointArray[3] = -0.025;
            swPointArray[4] = 0;
            swPointArray[5] = 0;

            swFeatMgr = Part.FeatureManager;

            swFeatData = (TablePatternFeatureData)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmTablePattern);
            swFeatData.GeometryPattern = false;
            swFeatData.PointArray = swPointArray;
            swFeatData.PropagateVisualProperty = true;
            swFeatData.UseCentroid = true;
            swFeat = swFeatMgr.CreateFeature(swFeatData);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Tangent Arc in a Sketch Example (C#)

This example shows how to create a tangent arc in a sketch.

```
//-----  
// Preconditions: Verify that the specified template exists.  
//  
// Postconditions:  
// 1. Creates a new part with a sketch of a tangent arc.  
// 2. Examine the graphics area.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace CreateTangentArc_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 Part;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS  
2017\\templates\\Part.prtdot", 0, 0, 0);  
  
            Part.SketchManager.InsertSketch(true);  
  
            object skSegment = null;  
  
            skSegment = Part.SketchManager.CreateLine(-0.060928,  
0.026745, 0.0, 0.06209, 0.002933, 0.0);  
  
            Part.ClearSelection2(true);  
  
            skSegment = Part.SketchManager.CreateTangentArc(0.06209,  
0.002933, 0.0, 0.020571, -0.021799, 0.0, 1);  
  
            Part.ClearSelection2(true);  
  
            Part.SketchManager.InsertSketch(true);  
  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Create Tangent Arc in a Sketch Example (VBA)

This example shows how to create a tangent arc in a sketch.

'-----
' Preconditions: Verify that the specified template exists.
,

' Postconditions:
' 1. Creates a new part with a sketch of a tangent arc.
' 2. Examine the graphics area.
'-----

```
Dim swApp As SldWorks.SldWorks
```

```
Dim Part As SldWorks.ModelDoc2
```

```
Option Explicit
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
```

```
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
```

```
    Set Part = swApp.ActiveDoc
```

```
    Part.SketchManager.InsertSketch True
```

```
    Dim skSegment As Object
```

```
    Set skSegment = Part.SketchManager.CreateLine(-0.060928, 0.026745, 0#, 0.06209, 0.002933, 0#)
```

```
    Part.ClearSelection2 True
```

```
    Set skSegment = Part.SketchManager.CreateTangentArc(0.06209, 0.002933, 0#, 0.020571, -0.021799, 0#, 1)
```

```
    Part.ClearSelection2 True
```

```
    Part.SketchManager.InsertSketch True
```

```
End Sub
```

Create Tangent Arc in a Sketch Example (VB.NET)

This example shows how to create a tangent arc in a sketch.

```
'-----  
' Preconditions: Verify that the specified template exists.  
'
```

```
' Postconditions:  
' 1. Creates a new part with a sketch of a tangent arc.  
' 2. Examine the graphics area.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Sub main()
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)  
        Part = swApp.ActiveDoc
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        Dim skSegment As Object
```

```
        skSegment = Part.SketchManager.CreateLine(-0.060928, 0.026745, 0.0#, 0.06209, 0.002933, 0.0#)
```

```
        Part.ClearSelection2(True)
```

```
        skSegment = Part.SketchManager.CreateTangentArc(0.06209, 0.002933, 0.0#, 0.020571, -0.021799, 0.0#, 1)
```

```
        Part.ClearSelection2(True)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Task Pane View Add-in Example (C#)

This example shows how to create an add-in that creates a Task Pane view and notifies when the Task Pane view and Display Pane are exercised.

//-----

1. Install the SOLIDWORKS API SDK templates using **SOLIDWORKS API SDK.msi** located in the apisd folder of your SOLIDWORKS installation. If apisd does not exist, modify your SOLIDWORKS Design installation from **Settings > Apps** (under Product Select, check **API Tools** and **SOLIDWORKS API SDK (install manually)**). **apisd/SOLIDWORKS API SDK.msi** will be placed in your installation folder.
2. Create a new project in Visual Studio using the SwCSharpAddin template.
3. Copy the following sample code into **SwAddin.cs** and **EventHandling.cs** of the new project.
4. Replace the Guid with a newly generated unique one if required (**VS menu > Tools > Create GUID**).
5. Add references to and modify the framework and build configuration of the project as needed.
6. Debug the program.

//-----

//SwAddin.cs

```
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;
using System.Collections.Generic;
using System.Diagnostics;

namespace SwCSharpAddin1
{
    /// <summary>
    /// Summary description for SwCSharpAddin1.
    /// </summary>
    [Guid("d67ed0ef-3bc7-4f13-b3c8-2469ad715ea0"), ComVisible(true)]
    [SwAddin(
        Description = "SwCSharpAddin1 description",
        Title = "SwCSharpAddin1",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp = null;
        ICommandManager iCmdMgr = null;
        int addinID = 0;
        TaskpaneView ctrl;

        public const int mainCmdGroupID = 5;
        public const int mainItemID1 = 0;
        public const int mainItemID2 = 1;
        public const int mainItemID3 = 2;
        public const int flyoutGroupID = 91;

        #region Event Handler Variables
        Hashtable openDocs = new Hashtable();
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr = null;
        #endregion

        #region Property Manager Variables
        UserPMPage ppage = null;
        #endregion
    }
}
```

```

// Public Properties
public ISldWorks SwApp
{
    get { return iSwApp; }
}

public ICommandManager CmdMgr
{
    get { return iCmdMgr; }
}

public Hashtable OpenDocs
{
    get { return openDocs; }
}

public int Cookie
{
    get { return addinID; }
}

#endregion

#region SolidWorks Registration
[ComRegisterFunctionAttribute]
public static void RegisterFunction ( Type t )
{
    #region Get Custom Attribute: SwAddinAttribute
    SwAddinAttribute SWattr = null;
    Type type = typeof(SwAddin);

    foreach (System.Attribute attr in type.GetCustomAttributes(false))
    {
        if (attr is SwAddinAttribute)
        {
            SWattr = attr as SwAddinAttribute;
            break;
        }
    }
}

#endregion

try
{
    Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
    Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

    string keyname = "SOFTWARE\\SolidWorks\\Addins\\" + t.GUID.ToString() + ";";
    Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
    addinkey.SetValue(null, 0);

    addinkey.SetValue("Description", SWattr.Description);
    addinkey.SetValue("Title", SWattr.Title);

    keyname = "Software\\SolidWorks\\AddInsStartup\\" + t.GUID.ToString() + ";";
    addinkey = hkcu.CreateSubKey(keyname);
    addinkey.SetValue(null, Convert.ToInt32(SWattr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
}
catch (System.NullReferenceException nl)
{
    Console.WriteLine("There was a problem registering this dll: SWattr is null. \n\" + nl.Message + "\"");
    System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: SWattr is null.\n\" + nl.Message +
    "\"");
}

```

```

        catch (System.Exception e)
        {
            Console.WriteLine(e.Message);

            System.Windows.Forms.MessageBox.Show("There was a problem registering the function: \n\"" + e.Message + "\"");
        }
    }

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction ( Type t )
{
    try
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

        string keyname = "SOFTWARE\\SolidWorks\\Addins\\" + t.GUID.ToString() + ";";
        hklm.DeleteSubKey(keyname);

        keyname = "Software\\SolidWorks\\AddInsStartup\\" + t.GUID.ToString() + ";";
        hkcu.DeleteSubKey(keyname);
    }
    catch (System.NullReferenceException nl)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + nl.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\"" + nl.Message + "\"");
    }
    catch (System.Exception e)
    {
        Console.WriteLine("There was a problem unregistering this dll: " + e.Message);
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: \n\"" + e.Message + "\"");
    }
}

#endregion

#region ISwAddin Implementation
public SwAddin ( )
{
}

public bool ConnectToSW ( object ThisSW, int cookie )
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Setup callbacks
    iSwApp.SetAddinCallbackInfo(0, this, addinID);

    // Register the callback for use with ISldWorks.PostMessageToApplication.
    iSwApp.AddCallback(addinID, "AddInMessageCallback");

    string[] bitmap = new string[1];
    string tooltip;

    bitmap[0] = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2024\\samples\\tutorial\\localhelp.bmp";
    tooltip = "Extruded block tab";
    ctrl = iSwApp.CreateTaskpaneView3(bitmap, tooltip);

    #region Setup the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();

```

```

        #endregion

        #region Setup Sample Property Manager
        AddPMP();
        #endregion

        return true;
    }

public bool DisconnectFromSW ( )
{
    RemovePMP();
    DetachEventHandlers();

    iSwApp.RemoveCallback(addinID);

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp);
    iSwApp = null;

    //The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect();
    GC.WaitForPendingFinalizers();

    GC.Collect();
    GC.WaitForPendingFinalizers();

    return true;
}
#endregion

// Ensure the dispatch pointer is marshalled correctly from the unmanaged world into the managed world.
public bool AddInMessageCallback ( int cmd, int data, [MarshalAs(UnmanagedType.IDispatch)] object dsp )
{
    if (cmd != (int)swAppCallBackCmd_e.swAppPostNotifyEvent)
    {
        return (false);
    }

    // Check we are the intended recipient, by checking cookie value.
    Debug.Assert(data == addinID);

    AddinCallbackData callbackdata = (AddinCallbackData)dsp;

    Trace.WriteLine("AddInMessageCallback called");
    Trace.WriteLine(" operation = " + callbackdata.Operation.ToString());
    Trace.WriteLine(" payload = " + callbackdata.Payload);

    switch (callbackdata.Operation)
    {
        case AddinCallbackData.swCallbackOperation_e.swDoWork:

            callbackdata.Handler.DoWorkCallback(callbackdata.Payload);

            break;

        case AddinCallbackData.swCallbackOperation_e.swDoMoreWork:

            callbackdata.Handler.DoMoreWorkCallback(callbackdata.Payload + "; Added some more work");

            break;

        default:

            break;
    }
}

```

```

        // Cleanup: account for the call to Marshal.GetIDispatchForObject().
        IntPtr pCallbackData = Marshal.GetIDispatchForObject(callbackdata);

        Int32 refcnt = Marshal.Release(pCallbackData);

        return (true);
    }

    #region UI Methods
    public void AddCommandMgr ( )
    {

    }

    public void RemoveCommandMgr ( )
    {

    }

    public Boolean AddPMP ( )
    {
        ppage = new UserPMPPage(this);
        return true;
    }

    public Boolean RemovePMP ( )
    {
        ppage = null;
        return true;
    }

    #endregion

    #region UI Callbacks
    public void CreateCube ( )
    {
        //make sure we have a part open
        string partTemplate = iSwApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
        if ((partTemplate != null) && (partTemplate != ""))
        {
            IModelDoc2 modDoc = (IModelDoc2)iSwApp.NewDocument(partTemplate, (int)swDwgPaperSizes_e.swDwgPaperA2size, 0.0,
0.0);

            modDoc.InsertSketch2(true);
            modDoc.SketchRectangle(0, 0, 0, .1, .1, .1, false);
            //Extrude the sketch
            IFeatureManager featMan = modDoc.FeatureManager;
            featMan.FeatureExtrusion(true,
                false, false,
                (int)swEndConditions_e.swEndCondBlind, (int)swEndConditions_e.swEndCondBlind,
                0.1, 0.0,
                false, false,
                false, false,
                0.0, 0.0,
                false, false,
                false, false,
                true,
                false, false);
        }
        else
        {
            System.Windows.Forms.MessageBox.Show("There is no part template available. Please check your options and make sure
there is a part template selected, or select a new part template.");
        }
    }
}

```

```

    }
}

public void ShowPMP ( )
{
    if (ppage != null)
        ppage.Show();
}

public int EnablePMP ( )
{
    if (iSwApp.ActiveDoc != null)
        return 1;
    else
        return 0;
}

#endregion

#region Event Methods
public bool AttachEventHandlers ( )
{
    AttachSwEvents();
    //Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents ( )
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);

        SwEventPtr.TaskPaneShowNotify += new DSldWorksEvents_TaskPaneShowNotifyEventHandler(TaskpaneShowNotify);
        SwEventPtr.TaskPaneHideNotify += new DSldWorksEvents_TaskPaneHideNotifyEventHandler(TaskpaneHideNotify);
        SwEventPtr.TaskPanePinnedNotify += new DSldWorksEvents_TaskPanePinnedNotifyEventHandler(TaskpanePinnedNotify);
        SwEventPtr.TaskPaneUnpinnedNotify += new
DSldWorksEvents_TaskPaneUnpinnedNotifyEventHandler(TaskpaneUnpinnedNotify);
        SwEventPtr.TaskPaneExpandNotify += new DSldWorksEvents_TaskPaneExpandNotifyEventHandler(TaskpaneExpandNotify);
        SwEventPtr.TaskPaneCollapseNotify += new
DSldWorksEvents_TaskPaneCollapseNotifyEventHandler(TaskpaneCollapseNotify);
        SwEventPtr.DisplayPaneActivationNotify += new
DSldWorksEvents_DisplayPaneActivationNotifyEventHandler(DisplayPaneActivationNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents ( )
{
    try
    {

```

```

        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);

        SwEventPtr.TaskPaneShowNotify += new DSldWorksEvents_TaskPaneShowNotifyEventHandler(TaskpaneShowNotify);
        SwEventPtr.TaskPaneHideNotify += new DSldWorksEvents_TaskPaneHideNotifyEventHandler(TaskpaneHideNotify);
        SwEventPtr.TaskPanePinnedNotify += new DSldWorksEvents_TaskPanePinnedNotifyEventHandler(TaskpanePinnedNotify);
        SwEventPtr.TaskPaneUnpinnedNotify += new
DSldWorksEvents_TaskPaneUnpinnedNotifyEventHandler(TaskpaneUnpinnedNotify);
        SwEventPtr.TaskPaneExpandNotify += new DSldWorksEvents_TaskPaneExpandNotifyEventHandler(TaskpaneExpandNotify);
        SwEventPtr.TaskPaneCollapseNotify += new
DSldWorksEvents_TaskPaneCollapseNotifyEventHandler(TaskpaneCollapseNotify);
        SwEventPtr.DisplayPaneActivationNotify += new
DSldWorksEvents_DisplayPaneActivationNotifyEventHandler(DisplayPaneActivationNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments ( )
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler ( ModelDoc2 modDoc )
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
            {
                docHandler = new PartEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocASSEMBLY:
            {
                docHandler = new AssemblyEventHandler(modDoc, this);
                break;
            }
            case (int)swDocumentTypes_e.swDocDRAWING:
            {
                docHandler = new DrawingEventHandler(modDoc, this);
                break;
            }
        }
    }
}

```

```

        default:
        {
            return false; //Unsupported document type
        }
    }
    docHandler.AttachEventHandlers();
    openDocs.Add(modDoc, docHandler);
}
return true;
}

public bool DetachModelEventHandler ( ModelDoc2 modDoc )
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers ( )
{
    DetachSwEvents();

    //Close events on all currently open docs
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    //Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        docHandler.DetachEventHandlers(); //This also removes the pair from the hash
        docHandler = null;
    }
    return true;
}
}
#endregion

#region Event Handlers
//Events
public int OnDocChange ( )
{
    return 0;
}

public int OnDocLoad ( string docTitle, string docPath )
{
    return 0;
}

int FileOpenPostNotify ( string FileName )
{
    AttachEventsToAllDocuments();
    return 0;
}

int TaskpaneShowNotify()
{
    bool isVisible = iSwApp.IsTaskPaneVisible();
    Trace.WriteLine("Addin received - TaskpaneShowNotify");
    return 0;
}

```

```

    }
    int TaskpaneHideNotify()
    {
        bool isVisible = iSwApp.IsTaskPaneVisible();
        Trace.WriteLine("Addin received - TaskpaneHideNotify");
        return 0;
    }
    int TaskpanePinnedNotify()
    {
        bool isVisible = iSwApp.IsTaskPaneVisible();
        bool isActive = ctrl.IsActiveTab();
        Trace.WriteLine("Addin received - TaskpanePinnedNotify");
        return 0;
    }
    int TaskpaneUnpinnedNotify()
    {
        Trace.WriteLine("Addin received - TaskpaneUnpinnedNotify");
        return 0;
    }

    int TaskpaneExpandNotify()
    {
        bool isExpanded = iSwApp.IsTaskPaneExpanded();
        Trace.WriteLine("Addin received - TaskpaneExpandNotify");
        return 0;
    }
    int TaskpaneCollapseNotify()
    {
        bool isExpanded = iSwApp.IsTaskPaneExpanded();
        Trace.WriteLine("Addin received - TaskpaneCollapseNotify");
        return 0;
    }

    int DisplayPaneActivationNotify(int ActivePaneIndex)
    {
        if(ActivePaneIndex == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
            MessageBox.Show("Nothing active");
        else if(ActivePaneIndex == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
            MessageBox.Show("Life Cycle active");
        else if(ActivePaneIndex == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
            MessageBox.Show("Display Pane active");
        return 0;
    }

    public int OnFileNew ( object newDoc, int docType, string templateName )
    {
        AttachEventsToAllDocuments();
        return 0;
    }

    public int OnModelChange ( )
    {
        return 0;
    }

    #endregion
}

}

//EventHandling.cs
using System;
using System.Collections;

//using SolidWorks.Interop.sldworks;
//using SolidWorks.Interop.swpublished;
//using SolidWorks.Interop.swconst;

```

```

using System.Runtime.InteropServices;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Windows.Forms;

namespace SwCSharpAddin1
{
    public class DocumentEventHandler
    {
        protected ISldWorks iSwApp;
        protected ModelDoc2 document;
        protected SwAddin userAddin;

        protected Hashtable openModelViews;

        public DocumentEventHandler ( ModelDoc2 modDoc, SwAddin addin )
        {
            document = modDoc;
            userAddin = addin;
            iSwApp = (ISldWorks)userAddin.SwApp;
            openModelViews = new Hashtable();
        }

        virtual public bool DoWorkCallback ( string strDescription)
        {
            // No-op. To be overridden in derived class.

            return (false);
        }

        virtual public bool DoMoreWorkCallback ( string strDescription )
        {
            // No-op. To be overridden in derived class.

            return (false);
        }

        virtual public bool AttachEventHandlers ( )
        {
            return true;
        }

        virtual public bool DetachEventHandlers ( )
        {
            return true;
        }

        public bool ConnectModelViews ( )
        {
            IModelView mView;
            mView = (IModelView)document.GetFirstModelView();

            while (mView != null)
            {
                if (!openModelViews.Contains(mView))
                {
                    DocView dView = new DocView(userAddin, mView, this);
                    dView.AttachEventHandlers();
                    openModelViews.Add(mView, dView);
                }
                mView = (IModelView)mView.GetNext();
            }
        }
    }
}

```

```

        return true;
    }

    public bool DisconnectModelViews ( )
    {
        //Close events on all currently open docs
        DocView dView;
        int numKeys;
        numKeys = openModelViews.Count;

        if (numKeys == 0)
        {
            return false;
        }

        object[] keys = new object[numKeys];

        //Remove all ModelView event handlers
        openModelViews.Keys.CopyTo(keys, 0);
        foreach (ModelView key in keys)
        {
            dView = (DocView)openModelViews[key];
            dView.DetachEventHandlers();
            openModelViews.Remove(key);
            dView = null;
        }
        return true;
    }

    public bool DetachModelViewEventHandler ( ModelView mView )
    {
        DocView dView;
        if (openModelViews.Contains(mView))
        {
            dView = (DocView)openModelViews[mView];
            openModelViews.Remove(mView);
            mView = null;
            dView = null;
        }
        return true;
    }
}

public class PartEventHandler : DocumentEventHandler
{
    PartDoc doc;

    public PartEventHandler ( ModelDoc2 modDoc, SwAddin addin )
        : base(modDoc, addin)
    {
        doc = (PartDoc)document;
    }

    override public bool DoWorkCallback ( string strDescription )
    {
        // Do some real work here.

        Trace.WriteLine("Processing posted message in Part.DoWorkCallback");
        Trace.WriteLine("  decription = " + strDescription);

        return (true);
    }
}

```

```

override public bool DoMoreWorkCallback ( string strDescription )
{
    // Do some real work here.

    Trace.WriteLine("Processing posted message in Part.DoMoreWorkCallback");
    Trace.WriteLine("  decription = " + strDescription);

    return (true);
}

override public bool AttachEventHandlers ( )
{
    doc.DestroyNotify += new DPartDocEvents_DestroyNotifyEventHandler(OnDestroy);
    doc.NewSelectionNotify += new DPartDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);
    doc.ModifyNotify += new DPartDocEvents_ModifyNotifyEventHandler(OnModify);

    doc.DisplayPaneExpandNotify += new DPartDocEvents_DisplayPaneExpandNotifyEventHandler(OnDisplayPaneExpandNotify);
    doc.DisplayPaneCollapseNotify += new DPartDocEvents_DisplayPaneCollapseNotifyEventHandler(OnDisplayPaneCollapseNotify);
    ConnectModelViews();

    return true;
}

override public bool DetachEventHandlers ( )
{
    doc.DestroyNotify -= new DPartDocEvents_DestroyNotifyEventHandler(OnDestroy);
    doc.NewSelectionNotify -= new DPartDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);
    doc.ModifyNotify -= new DPartDocEvents_ModifyNotifyEventHandler(OnModify);

    doc.DisplayPaneExpandNotify -= new DPartDocEvents_DisplayPaneExpandNotifyEventHandler(OnDisplayPaneExpandNotify);
    doc.DisplayPaneCollapseNotify -= new DPartDocEvents_DisplayPaneCollapseNotifyEventHandler(OnDisplayPaneCollapseNotify);

    DisconnectModelViews();

    userAddin.DetachModelEventHandler(document);
    return true;
}

//Event Handlers
public int OnDestroy ( )
{
    DetachEventHandlers();
    return 0;
}

public int OnNewSelection ( )
{
    return 0;
}

public int OnModify ( )
{
    Trace.WriteLine("Deferring work in PartDoc.OnModify");

    AddinCallbackData callbackdata = new AddinCallbackData(AddinCallbackData.swCallbackOperation_e.swDoWork, this, "Work on
this. Greetings from PartDoc.OnModify");

    // The returned pointer is one to the CCW for the managed object.
    // That unmanaged CCW is referenced counted in COM style.
    // So we assume it will be alive and in the same memory location for the duration of the call back.
    IntPtr pCallbackData = Marshal.GetIDispatchForObject(callbackdata);

    if (Win32.Is64Bit())
    {
        iSwApp.PostMessageToApplicationx64(userAddin.Cookie, pCallbackData.ToInt64());
    }
}

```

```

    }
    else
    {
        iSwApp.PostMessageToApplication(userAddin.Cookie, pCallbackData.ToInt32());
    }

    return 0;
}

public int OnDisplayPaneExpandNotify()
{
    ModelDocExtension docExtn;
    docExtn = document.Extension;

    bool isVisible = docExtn.IsDisplayPaneVisible();

    int index = iSwApp.GetActiveDisplayPane();
    if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
        MessageBox.Show("Nothing active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
        MessageBox.Show("Life Cycle active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
        MessageBox.Show("Display Pane active");
    return 0;
}

public int OnDisplayPaneCollapseNotify()
{
    ModelDocExtension docExtn;
    docExtn = document.Extension;

    bool isVisible = docExtn.IsDisplayPaneVisible();

    int index = iSwApp.GetActiveDisplayPane();
    if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
        MessageBox.Show("Nothing active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
        MessageBox.Show("Life Cycle active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
        MessageBox.Show("Display Pane active");
    return 0;
}
}

public class AssemblyEventHandler : DocumentEventHandler
{
    AssemblyDoc doc;
    SwAddin swAddin;

    public AssemblyEventHandler ( ModelDoc2 modDoc, SwAddin addin )
        : base(modDoc, addin)
    {
        doc = (AssemblyDoc)document;
        swAddin = addin;
    }

    override public bool DoWorkCallback ( string strDescription )
    {
        // Do some real work here.

        Trace.WriteLine("Processing posted message in Assembly.DoWorkCallback");
        Trace.WriteLine("  decription = " + strDescription);

        return (true);
    }
}

```

```

override public bool DoMoreWorkCallback ( string strDescription )
{
    // Do some real work here.

    Trace.WriteLine("Processing posted message in Assembly.DoMoreWorkCallback");
    Trace.WriteLine("  decription = " + strDescription);

    return (true);
}

override public bool AttachEventHandlers ( )
{
    doc.DestroyNotify += new DAssemblyDocEvents_DestroyNotifyEventHandler(OnDestroy);
    doc.NewSelectionNotify += new DAssemblyDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);
    doc.ComponentStateChangeNotify2 += new
DAssemblyDocEvents_ComponentStateChangeNotify2EventHandler(ComponentStateChangeNotify2);
    doc.ComponentStateChangeNotify += new
DAssemblyDocEvents_ComponentStateChangeNotifyEventHandler(ComponentStateChangeNotify);
    doc.ComponentVisualPropertiesChangeNotify += new
DAssemblyDocEvents_ComponentVisualPropertiesChangeNotifyEventHandler(ComponentVisualPropertiesChangeNotify);
    doc.ComponentDisplayStateChangeNotify += new
DAssemblyDocEvents_ComponentDisplayStateChangeNotifyEventHandler(ComponentDisplayStateChangeNotify);
    doc.ModifyNotify += new DAssemblyDocEvents_ModifyNotifyEventHandler(OnModify);

    doc.DisplayPaneExpandNotify += new DAssemblyDocEvents_DisplayPaneExpandNotifyEventHandler(OnDisplayPaneExpandNotify);
    doc.DisplayPaneCollapseNotify += new
DAssemblyDocEvents_DisplayPaneCollapseNotifyEventHandler(OnDisplayPaneCollapseNotify);

    ConnectModelViews();

    return true;
}

override public bool DetachEventHandlers ( )
{
    doc.DestroyNotify -= new DAssemblyDocEvents_DestroyNotifyEventHandler(OnDestroy);
    doc.NewSelectionNotify -= new DAssemblyDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);
    doc.ComponentStateChangeNotify2 -= new
DAssemblyDocEvents_ComponentStateChangeNotify2EventHandler(ComponentStateChangeNotify2);
    doc.ComponentStateChangeNotify -= new
DAssemblyDocEvents_ComponentStateChangeNotifyEventHandler(ComponentStateChangeNotify);
    doc.ComponentVisualPropertiesChangeNotify -= new
DAssemblyDocEvents_ComponentVisualPropertiesChangeNotifyEventHandler(ComponentVisualPropertiesChangeNotify);
    doc.ComponentDisplayStateChangeNotify -= new
DAssemblyDocEvents_ComponentDisplayStateChangeNotifyEventHandler(ComponentDisplayStateChangeNotify);
    doc.ModifyNotify -= new DAssemblyDocEvents_ModifyNotifyEventHandler(OnModify);

    doc.DisplayPaneExpandNotify -= new DAssemblyDocEvents_DisplayPaneExpandNotifyEventHandler(OnDisplayPaneExpandNotify);
    doc.DisplayPaneCollapseNotify -= new
DAssemblyDocEvents_DisplayPaneCollapseNotifyEventHandler(OnDisplayPaneCollapseNotify);

    DisconnectModelViews();

    userAddin.DetachModelEventHandler(document);
    return true;
}

//Event Handlers
public int OnDestroy ( )
{
    DetachEventHandlers();
    return 0;
}

```

```

public int OnNewSelection ( )
{
    return 0;
}

public int OnModify ( )
{
    Trace.WriteLine("Deferring work in AssemblyDoc.OnModify");

    AddinCallbackData callbackdata = new AddinCallbackData(AddinCallbackData.swCallbackOperation_e.swDoMoreWork, this,
"Work on this. Greetings from AssemblyDoc.OnModify");

    // The returned pointer is one to the CCW for the managed object.
    // That unmanaged CCW is referenced counted in COM style.
    // So we assume it will be alive and in the same memory location for the duration of the call back.
    IntPtr pCallbackData = Marshal.GetIDispatchForObject(callbackdata);

    if (Win32.Is64Bit())
    {
        iSwApp.PostMessageToApplicationx64(userAddin.Cookie, pCallbackData.ToInt64());
    }
    else
    {
        iSwApp.PostMessageToApplication(userAddin.Cookie, pCallbackData.ToInt32());
    }

    return 0;
}

public int OnDisplayPaneExpandNotify()
{
    ModelDocExtension docExtn;
    docExtn = document.Extension;

    bool isvisible = docExtn.IsDisplayPaneVisible();

    int index = iSwApp.GetActiveDisplayPane();
    if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
        MessageBox.Show("Nothing active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
        MessageBox.Show("Life Cycle active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
        MessageBox.Show("Display Pane active");
    return 0;
}

public int OnDisplayPaneCollapseNotify()
{
    ModelDocExtension docExtn;
    docExtn = document.Extension;

    bool isvisible = docExtn.IsDisplayPaneVisible();

    int index = iSwApp.GetActiveDisplayPane();
    if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
        MessageBox.Show("Nothing active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
        MessageBox.Show("Life Cycle active");
    else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
        MessageBox.Show("Display Pane active");
    return 0;
}

```

```

//attach events to a component if it becomes resolved
protected int ComponentStateChange ( object componentModel, short newCompState )
{
    ModelDoc2 modDoc = (ModelDoc2)componentModel;
    swComponentSuppressionState_e newState = (swComponentSuppressionState_e)newCompState;

    switch (newState)
    {
        case swComponentSuppressionState_e.swComponentFullyResolved:
            {
                if ((modDoc != null) & !this.swAddin.OpenDocs.Contains(modDoc))
                {
                    this.swAddin.AttachModelDocEventHandler(modDoc);
                }
                break;
            }

        case swComponentSuppressionState_e.swComponentResolved:
            {
                if ((modDoc != null) & !this.swAddin.OpenDocs.Contains(modDoc))
                {
                    this.swAddin.AttachModelDocEventHandler(modDoc);
                }
                break;
            }
    }
    return 0;
}

protected int ComponentStateChange ( object componentModel )
{
    ComponentStateChange(componentModel, (short)swComponentSuppressionState_e.swComponentResolved);
    return 0;
}

public int ComponentStateChangeNotify2 ( object componentModel, string CompName, short oldCompState, short newCompState )
{
    return ComponentStateChange(componentModel, newCompState);
}

int ComponentStateChangeNotify ( object componentModel, short oldCompState, short newCompState )
{
    return ComponentStateChange(componentModel, newCompState);
}

int ComponentDisplayStateChangeNotify ( object swObject )
{
    Component2 component = (Component2)swObject;
    ModelDoc2 modDoc = (ModelDoc2)component.GetModelDoc();

    return ComponentStateChange(modDoc);
}

int ComponentVisualPropertiesChangeNotify ( object swObject )
{
    Component2 component = (Component2)swObject;
    ModelDoc2 modDoc = (ModelDoc2)component.GetModelDoc();

    return ComponentStateChange(modDoc);
}
}

```

```

public class DrawingEventHandler : DocumentEventHandler
{
    DrawingDoc doc;

    public DrawingEventHandler ( ModelDoc2 modDoc, SwAddin addin )
        : base(modDoc, addin)
    {
        doc = (DrawingDoc)document;
    }

    override public bool AttachEventHandlers ( )
    {
        doc.DestroyNotify += new DDrawingDocEvents_DestroyNotifyEventHandler(OnDestroy);
        doc.NewSelectionNotify += new DDrawingDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);

        doc.DisplayPaneExpandNotify += new DDrawingDocEvents_DisplayPaneExpandNotifyEventHandler(OnDisplayPaneExpandNotify);
        doc.DisplayPaneCollapseNotify += new
DDrawingDocEvents_DisplayPaneCollapseNotifyEventHandler(OnDisplayPaneCollapseNotify);

        ConnectModelViews();

        return true;
    }

    public int OnDisplayPaneExpandNotify()
    {
        ModelDocExtension docExtn;
        docExtn = document.Extension;

        bool isvisible = docExtn.IsDisplayPaneVisible();

        int index = iSwApp.GetActiveDisplayPane();
        if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
            MessageBox.Show("Nothing active");
        else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
            MessageBox.Show("Life Cycle active");
        else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
            MessageBox.Show("Display Pane active");
        return 0;
    }

    public int OnDisplayPaneCollapseNotify()
    {
        ModelDocExtension docExtn;
        docExtn = document.Extension;

        bool isvisible = docExtn.IsDisplayPaneVisible();

        int index = iSwApp.GetActiveDisplayPane();
        if (index == (int)swDisplayPaneIndex_e.swDisplayPaneNone)
            MessageBox.Show("Nothing active");
        else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneLifeCycleTab)
            MessageBox.Show("Life Cycle active");
        else if (index == (int)swDisplayPaneIndex_e.swDisplayPaneTab)
            MessageBox.Show("Display Pane active");
        return 0;
    }

    override public bool DetachEventHandlers ( )
    {
        doc.DestroyNotify -= new DDrawingDocEvents_DestroyNotifyEventHandler(OnDestroy);
        doc.NewSelectionNotify -= new DDrawingDocEvents_NewSelectionNotifyEventHandler(OnNewSelection);
    }
}

```

```

        DisconnectModelViews();

        userAddin.DetachModelEventHandler(document);
        return true;
    }

    //Event Handlers
    public int OnDestroy ( )
    {
        DetachEventHandlers();
        return 0;
    }

    public int OnNewSelection ( )
    {
        return 0;
    }
}

public class DocView
{
    ISldWorks iSwApp;
    SwAddin userAddin;
    ModelView mView;
    DocumentEventHandler parent;

    public DocView ( SwAddin addin, IModelView mv, DocumentEventHandler doc )
    {
        userAddin = addin;
        mView = (ModelView)mv;
        iSwApp = (ISldWorks)userAddin.SwApp;
        parent = doc;
    }

    public bool AttachEventHandlers ( )
    {
        mView.DestroyNotify2 += new DModelViewEvents_DestroyNotify2EventHandler(OnDestroy);
        mView.RepaintNotify += new DModelViewEvents_RepaintNotifyEventHandler(OnRepaint);
        return true;
    }

    public bool DetachEventHandlers ( )
    {
        mView.DestroyNotify2 -= new DModelViewEvents_DestroyNotify2EventHandler(OnDestroy);
        mView.RepaintNotify -= new DModelViewEvents_RepaintNotifyEventHandler(OnRepaint);
        parent.DetachModelViewEventHandler(mView);
        return true;
    }

    //EventHandlers
    public int OnDestroy ( int destroyType )
    {
        return 0;
    }

    public int OnRepaint ( int repaintType )
    {
        return 0;
    }
}
}

```

Create Temporary Bodies by Offsetting a Surface Body Example (C#)

This example shows how to create two temporary bodies by offsetting a surface body.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Add a reference to Microsoft.VisualBasic (right-click the name of
// the project in the Project Explorer, click Add Reference, the .NET tab >
// Microsoft.VisualBasic > OK.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a surface body.
// 3. Selects an edge on the surface body to offset.
// 4. Creates two temporary bodies of the surface
// body using the selected edge.
// 5. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using Microsoft.VisualBasic;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatureManager = default(FeatureManager);
            SketchSegment sketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            Edge swEdge = default(Edge);
            Body2 swBody = default(Body2);
            Body2 newBody1 = default(Body2);
            Body2 newBody2 = default(Body2);
            object pointArray = null;
            double[] points = new double[12];
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;

            //Create extruded surface body
            points[0] = -0.0720746414289124;
            points[1] = -0.0283600245263074;
            points[2] = 0;
            points[3] = -0.0514715593755;
            points[4] = -0.00345025084396866;
            points[5] = 0;
            points[6] = 0;
            points[7] = 0;
            points[8] = 0;
            points[9] = 0.0872558597840225;
            points[10] = 0.0521037067517796;
            points[11] = 0;
            pointArray = points;
            sketchSegment = (SketchSegment)swSketchManager.CreateSpline((pointArray));
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);
            swFeatureManager.FeatureExtrudeRefSurface2(true, false, false, 0, 0, 0.0508, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, false, false,
            false);
            swSelectionManager.EnableContourSelection = false;

            //Offset selected edge and create two temporary bodies
            status = swModelDocExt.SelectByID2("", "EDGE", -0.00623752003605205, 0.000329492391927033, 0.050581684437077, false, 0, null, 0);
            swEdge = (Edge)swSelectionManager.GetSelectedObject6(1, -1);
            swBody = (Body2)swEdge.GetBody();
            swBody = (Body2)swBody.Copy();
            //Using a copy of the selected surface body, create two new temporary bodies
            newBody1 = (Body2)swBody.MakeOffset(0.01, false);
            newBody2 = (Body2)swBody.MakeOffset(0.01, true);
            //Display and color the new temporary body blue
            newBody1.Display3(swModel, Information.RGB(0, 0, 255), (int)swTempBodySelectOptions_e.swTempBodySelectOptionNone);
            //Display and color the new temporary body red
            newBody2.Display3(swModel, Information.RGB(255, 0, 0), (int)swTempBodySelectOptions_e.swTempBodySelectOptionNone);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Temporary Bodies by Offsetting a Surface Body Example (VBA)

This example shows how to create two temporary bodies by offsetting a surface body.

```
'-----
' Preconditions: Verify that the specified part document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a surface body.
' 3. Selects an edge on the surface body to offset.
' 4. Creates two temporary bodies of the surface
'    body using the selected edge.
' 5. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatureManager As SldWorks.FeatureManager
Dim sketchSegment As SldWorks.sketchSegment
Dim swSketchManager As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swEdge As SldWorks.Edge
Dim swBody As SldWorks.Body2
Dim newBody1 As SldWorks.Body2
Dim newBody2 As SldWorks.Body2
Dim pointArray As Variant
Dim points(11) As Double
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swSketchManager = swModel.SketchManager
    Set swModelDocExt = swModel.Extension
    Set swSelectionManager = swModel.SelectionManager

    'Create extruded surface body
    points(0) = -7.20746414289124E-02
    points(1) = -2.83600245263074E-02
    points(2) = 0
    points(3) = -0.0514715593755
    points(4) = -3.45025084396866E-03
    points(5) = 0
    points(6) = 0
    points(7) = 0
    points(8) = 0
    points(9) = 8.72558597840225E-02
    points(10) = 5.21037067517796E-02
    points(11) = 0
    pointArray = points
    Set sketchSegment = swSketchManager.CreateSpline((pointArray))
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    swFeatureManager.FeatureExtruRefSurface2 True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, False, False, False
    swSelectionManager.EnableContourSelection = False

    'Offset selected edge and create two temporary bodies
    status = swModelDocExt.SelectByID2("", "EDGE", -6.23752003605205E-03, 3.29492391927033E-04, 0.050581684437077, False, 0, Nothing, 0)
    Set swEdge = swSelectionManager.GetSelectedObject6(1, -1)
    Set swBody = swEdge.GetBody
    Set swBody = swBody.Copy
    'Using a copy of the selected surface body, create two new temporary bodies
    Set newBody1 = swBody.MakeOffset(0.01, False)
    Set newBody2 = swBody.MakeOffset(0.01, True)
    'Display and color the new temporary body blue
    newBody1.Display3 swModel, RGB(0, 0, 255), swTempbodySelectOptions_e.swTempBodySelectOptionNone
    'Display and color the new temporary body red
    newBody2.Display3 swModel, RGB(255, 0, 0), swTempbodySelectOptions_e.swTempBodySelectOptionNone

End Sub
```

Create Temporary Bodies by Offsetting a Surface Body Example (VB.NET)

This example shows how to create two temporary bodies by offsetting a surface body.

```

'-----
' Preconditions: Verify that the specified part document template exists.
'-----
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a surface body.
' 3. Selects an edge on the surface body to offset.
' 4. Creates two temporary bodies of the surface
'    body using the selected edge.
' 5. Examine the graphics area.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatureManager As FeatureManager
        Dim sketchSegment As SketchSegment
        Dim swSketchManager As SketchManager
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionManager As SelectionMgr
        Dim swEdge As Edge
        Dim swBody As Body2
        Dim newBody1 As Body2
        Dim newBody2 As Body2
        Dim pointArray As Object
        Dim points(11) As Double
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swFeatureManager = swModel.FeatureManager
        swSketchManager = swModel.SketchManager
        swModelDocExt = swModel.Extension
        swSelectionManager = swModel.SelectionManager

        'Create extruded surface body
        points(0) = -0.0720746414289124
        points(1) = -0.0283600245263074
        points(2) = 0
        points(3) = -0.0514715593755
        points(4) = -0.00345025084396866
        points(5) = 0
        points(6) = 0
        points(7) = 0
        points(8) = 0
        points(9) = 0.0872558597840225
        points(10) = 0.0521037067517796
        points(11) = 0
        pointArray = points
        sketchSegment = swSketchManager.CreateSpline((pointArray))
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeatureManager.FeatureExtrudeRefSurface2(True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 0.0174532925199433, 0.017453292519
9433, False, False, False, False, False, False, False)
        swSelectionManager.EnableContourSelection = False

        'Offset selected edge and create two temporary bodies
        status = swModelDocExt.SelectByID("", "EDGE", -0.00623752003605205, 0.000329492391927033, 0.050581684437077, False, 0, Nothing, 0)
        swEdge = swSelectionManager.GetSelectedObject6(1, -1)
        swBody = swEdge.GetBody
        swBody = swBody.Copy
        'Using a copy of the selected surface body, create two new temporary bodies
        newBody1 = swBody.MakeOffset(0.01, False)
        newBody2 = swBody.MakeOffset(0.01, True)
        'Display and color the new temporary body blue
        newBody1.Display3(swModel, RGB(0, 0, 255), swTempBodySelectOptions_e.swTempBodySelectOptionNone)
        'Display and color the new temporary body red
        newBody2.Display3(swModel, RGB(255, 0, 0), swTempBodySelectOptions_e.swTempBodySelectOptionNone)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create Temporary Elliptical Extrusion Example (C#)

This example shows how to create a temporary elliptical body.

```
//-----
// Preconditions:
// 1. Open a model document in SOLIDWORKS Design.
// 2. Open an Immediate Window.
//
// Postconditions:
// 1. Creates a temporary elliptical extruded body.
// 2. Inspect the Immediate Window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateTempExtrudedBody_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swDocument = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            Modeler swModeler = default(Modeler);

            Curve[] swCurve = new Curve[1];
            object vCenter = null;
            double dMajorRadius = 0;
            double dMinorRadius = 0;
            object vMajorAxis = null;
            object vMinorAxis = null;
            double[] vEllipseParams = null;
            double[] aCenter = new double[3];
            double[] aMajorAxis = new double[3];
            double[] aMinorAxis = new double[3];
            double dStartParam = 0;
            double dEndParam = 0;
            bool bIsClosed = false;
            bool bIsPeriodic = false;
            bool bStatus = false;

            swDocument = (ModelDoc2)swApp.ActiveDoc;

            if ((swDocument == null))
            {
                swDocument = (ModelDoc2)swApp.NewPart();
            }

            if ((swDocument.GetType() != (int)swDocumentTypes_e.swDocPART))
            {
                return;
            }

            swPart = (PartDoc)swDocument;
            swModeler = (Modeler)swApp.GetModeler();

            aCenter[0] = 0.0;
            aCenter[1] = 0.0;
            aCenter[2] = 0.0;

            vCenter = aCenter;

            dMajorRadius = 2.0;

            aMajorAxis[0] = 1.0;
            aMajorAxis[1] = 0.0;
            aMajorAxis[2] = 0.0;

            vMajorAxis = aMajorAxis;

            dMinorRadius = 1.0;

            aMinorAxis[0] = 0.0;
            aMinorAxis[1] = 1.0;
            aMinorAxis[2] = 0.0;

            vMinorAxis = aMinorAxis;

            swCurve[0] = (Curve)swModeler.CreateEllipse(vCenter, dMajorRadius, dMinorRadius, vMajorAxis, vMinorAxis);

            if ((swCurve[0] == null))
```

```

{
    Debug.Print(" No curve");
}
else
{
    Debug.Print("Curve:");
    Debug.Print(" is ellipse = " + (swCurve[0].IsEllipse() == false ? "False" : "True"));
    Debug.Print(" type = " + swCurve[0].Identity());
    Debug.Print(" is ellipse = " + (swCurve[0].Identity() == (int)swCurveTypes_e.ELLIPSE_TYPE));
    Debug.Print(" trimmed = " + (swCurve[0].IsTrimmedCurve() == false ? "False" : "True"));

    bStatus = swCurve[0].GetEndParams(out dStartParam, out dEndParam, out bIsClosed, out bIsPeriodic);

    Debug.Print(" start param = " + dStartParam);
    Debug.Print(" end param = " + dEndParam);
    Debug.Print(" closed = " + bIsClosed);
    Debug.Print(" periodic = " + bIsPeriodic);

    Debug.Print(" length = " + swCurve[0].GetLength3(dStartParam, dEndParam));

    if (!(swCurve[0].IsEllipse() == false))
    {
        vEllipseParams = (double[])swCurve[0].GetEllipseParams();

        Debug.Print(" centre = (" + vEllipseParams[0] + ", " + vEllipseParams[1] + ", " + vEllipseParams[2] +
        ");");
        Debug.Print(" major radius = " + vEllipseParams[3]);
        Debug.Print(" major axis = (" + vEllipseParams[4] + ", " + vEllipseParams[5] + ", " + vEllipseParams[6] +
        ");");
        Debug.Print(" minor radius = " + vEllipseParams[7]);
        Debug.Print(" minor axis = (" + vEllipseParams[8] + ", " + vEllipseParams[9] + ", " + vEllipseParams[10] +
        ");");
    }

    Surface planeSurf = default(Surface);
    MathUtility swMath = default(MathUtility);
    double slotDepth = 0;
    slotDepth = 0.01;

    swMath = (MathUtility)swApp.GetMathUtility();

    double[] startArr = new double[3];
    double[] endArr = new double[3];
    double[] ptArr = new double[3];
    double[] dirArr = new double[3];

    ptArr[0] = 0.0;
    ptArr[1] = 0.0;
    ptArr[2] = 0.0;
    dirArr[0] = 0.0;
    dirArr[1] = 0.0;
    dirArr[2] = 1.0;
    startArr[0] = 1.0;
    startArr[1] = 0.0;
    startArr[2] = 0.0;

    planeSurf = (Surface)swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr));

    Body2 profileBody = default(Body2);
    Body2 extrudedBody = default(Body2);
    MathVector dirVector = default(MathVector);

    profileBody = (Body2)planeSurf.CreateTrimmedSheet4((swCurve), true);

    dirArr[0] = 0.0;
    dirArr[1] = 0.0;
    dirArr[2] = -1.0;

    dirVector = (MathVector)swMath.CreateVector((dirArr));
    extrudedBody = swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth);
    extrudedBody.Display3(swDocument, Information.RGB(1, 0, 0), 0);
}
swDocument.ViewZoomtofit();
}

public SldWorks swApp;
}
}

```

Create Temporary Elliptical Extrusion Example (VBA)

This example shows how to create a temporary elliptical body.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Open an Immediate Window.
'
' Postconditions:
' 1. Creates a temporary elliptical extruded body.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swDocument As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim swModeler As SldWorks.Modeler
```

```
Dim swCurve(0) As SldWorks.Curve
Dim vCenter As Variant
Dim dMajorRadius As Double
Dim dMinorRadius As Double
Dim vMajorAxis As Variant
Dim vMinorAxis As Variant
Dim vEllipseParams As Variant
Dim aCenter(2) As Double
Dim aMajorAxis(2) As Double
Dim aMinorAxis(2) As Double
Dim dStartParam As Double
Dim dEndParam As Double
Dim bIsClosed As Boolean
Dim bIsPeriodic As Boolean
Dim bStatus As Boolean
```

```
Set swApp = Application.SldWorks
Set swDocument = swApp.ActiveDoc
```

```
If (swDocument Is Nothing) Then
    Set swDocument = swApp.NewPart
End If
```

```
If (swDocument.GetType <> swDocPART) Then
    Exit Sub
End If
```

```
Set swPart = swDocument
Set swModeler = swApp.GetModeler
```

```
aCenter(0) = 0#
aCenter(1) = 0#
aCenter(2) = 0#
```

```
vCenter = aCenter
```

```
dMajorRadius = 2#
```

```
aMajorAxis(0) = 1#
aMajorAxis(1) = 0#
aMajorAxis(2) = 0#
```

```
vMajorAxis = aMajorAxis
```

```
dMinorRadius = 1#
```

```
aMinorAxis(0) = 0#
aMinorAxis(1) = 1#
aMinorAxis(2) = 0#
```

```
vMinorAxis = aMinorAxis
```

```
Set swCurve(0) = swModeler.CreateEllipse(vCenter, dMajorRadius, dMinorRadius, vMajorAxis, vMinorAxis)
```

```
If (swCurve(0) Is Nothing) Then
    Debug.Print " No curve"
Else
```

```
    Debug.Print "Curve:"
    Debug.Print " is ellipse = " & IIf(swCurve(0).IsEllipse = False, "False", "True")
    Debug.Print " type = " & swCurve(0).Identity
```

```

Debug.Print " is ellipse = " & (swCurve(0).Identity = swCurveTypes_e.ELLIPSE_TYPE)
Debug.Print " trimmed = " & IIf(swCurve(0).IsTrimmedCurve = False, "False", "True")

bStatus = swCurve(0).GetEndParams(dStartParam, dEndParam, bIsClosed, bIsPeriodic)

Debug.Print " start param = " & dStartParam
Debug.Print " end param = " & dEndParam
Debug.Print " closed = " & bIsClosed
Debug.Print " periodic = " & bIsPeriodic

Debug.Print " length = " & swCurve(0).GetLength3(dStartParam, dEndParam)

If (Not (swCurve(0).IsEllipse = False)) Then
    vEllipseParams = swCurve(0).GetEllipseParams

    Debug.Print " centre = (" & vEllipseParams(0) & ", " & vEllipseParams(1) & ", " & vEllipseParams(2) & ")"
    Debug.Print " major radius = " & vEllipseParams(3)
    Debug.Print " major axis = (" & vEllipseParams(4) & ", " & vEllipseParams(5) & ", " & vEllipseParams(6) & ")"
    Debug.Print " minor radius = " & vEllipseParams(7)
    Debug.Print " minor axis = (" & vEllipseParams(8) & ", " & vEllipseParams(9) & ", " & vEllipseParams(10) & ")"
End If

Dim planeSurf As SldWorks.Surface
Dim swMath As SldWorks.MathUtility
Dim slotDepth As Double
slotDepth = 0.01

Set swMath = swApp.GetMathUtility

Dim startArr(2) As Double
Dim endArr(2) As Double
Dim ptArr(2) As Double
Dim dirArr(2) As Double

ptArr(0) = 0#
ptArr(1) = 0#
ptArr(2) = 0#
dirArr(0) = 0#
dirArr(1) = 0#
dirArr(2) = 1#
startArr(0) = 1#
startArr(1) = 0#
startArr(2) = 0#

Set planeSurf = swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr))

Dim profileBody As SldWorks.Body2
Dim extrudedBody As SldWorks.Body2
Dim dirVector As SldWorks.MathVector

Set profileBody = planeSurf.CreateTrimmedSheet4((swCurve), True)
dirArr(0) = 0#
dirArr(1) = 0#
dirArr(2) = -1#

Set dirVector = swMath.CreateVector((dirArr))
Set extrudedBody = swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth)
extrudedBody.Display3 swDocument, RGB(1, 0, 0), 0
swDocument.ViewZoomtofit

End If

End Sub

```

Create Temporary Elliptical Extrusion Example (VB.NET)

This example shows how to create a temporary elliptical body.

```

'-----
' Preconditions:
' 1. Open a model document in SOLIDWORKS Design.
' 2. Open an Immediate Window.
'
' Postconditions:
' 1. Creates a temporary elliptical extruded body.
' 2. Inspect the Immediate Window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub Main()
```

```
        Dim swDocument As ModelDoc2
        Dim swPart As PartDoc
        Dim swModeler As Modeler

```

```

        Dim swCurve(0) As Curve
        Dim vCenter As Object
        Dim dMajorRadius As Double
        Dim dMinorRadius As Double
        Dim vMajorAxis As Object
        Dim vMinorAxis As Object
        Dim vEllipseParams As Object
        Dim aCenter(2) As Double
        Dim aMajorAxis(2) As Double
        Dim aMinorAxis(2) As Double
        Dim dStartParam As Double
        Dim dEndParam As Double
        Dim bIsClosed As Boolean
        Dim bIsPeriodic As Boolean
        Dim bStatus As Boolean

```

```
        swDocument = swApp.ActiveDoc
```

```

        If (swDocument Is Nothing) Then
            swDocument = swApp.NewPart
        End If

```

```

        If (swDocument.GetType <> swDocumentTypes_e.swDocPART) Then
            Exit Sub
        End If

```

```

        swPart = swDocument
        swModeler = swApp.GetModeler

```

```

        aCenter(0) = 0.0#
        aCenter(1) = 0.0#
        aCenter(2) = 0.0#

```

```
        vCenter = aCenter
```

```
        dMajorRadius = 2.0#
```

```

        aMajorAxis(0) = 1.0#
        aMajorAxis(1) = 0.0#
        aMajorAxis(2) = 0.0#

```

```
        vMajorAxis = aMajorAxis
```

```
        dMinorRadius = 1.0#
```

```

        aMinorAxis(0) = 0.0#
        aMinorAxis(1) = 1.0#
        aMinorAxis(2) = 0.0#

```

```
        vMinorAxis = aMinorAxis
```

```
        swCurve(0) = swModeler.CreateEllipse(vCenter, dMajorRadius, dMinorRadius, vMajorAxis, vMinorAxis)
```

```

        If (swCurve(0) Is Nothing) Then
            Debug.Print(" No curve")
        Else

```

```

            Debug.Print("Curve:")
            Debug.Print(" is ellipse = " & IIf(swCurve(0).IsEllipse = False, "False", "True"))
            Debug.Print(" type = " & swCurve(0).Identity)
            Debug.Print(" is ellipse = " & (swCurve(0).Identity = swCurveTypes_e.ELLIPSE_TYPE))
            Debug.Print(" trimmed = " & IIf(swCurve(0).IsTrimmedCurve = False, "False", "True"))

```

```

bStatus = swCurve(0).GetEndParams(dStartParam, dEndParam, bIsClosed, bIsPeriodic)

Debug.Print(" start param = " & dStartParam)
Debug.Print(" end param = " & dEndParam)
Debug.Print(" closed = " & bIsClosed)
Debug.Print(" periodic = " & bIsPeriodic)

Debug.Print(" length = " & swCurve(0).GetLength3(dStartParam, dEndParam))

If (Not (swCurve(0).IsEllipse = False)) Then
    vEllipseParams = swCurve(0).GetEllipseParams

    Debug.Print(" centre = (" & vEllipseParams(0) & ", " & vEllipseParams(1) & ", " & vEllipseParams(2) & ")")
    Debug.Print(" major radius = " & vEllipseParams(3))
    Debug.Print(" major axis = (" & vEllipseParams(4) & ", " & vEllipseParams(5) & ", " & vEllipseParams(6) & ")")
    Debug.Print(" minor radius = " & vEllipseParams(7))
    Debug.Print(" minor axis = (" & vEllipseParams(8) & ", " & vEllipseParams(9) & ", " & vEllipseParams(10) & ")")
End If

Dim planeSurf As Surface
Dim swMath As MathUtility
Dim slotDepth As Double
slotDepth = 0.01

swMath = swApp.GetMathUtility

Dim startArr(2) As Double
Dim endArr(2) As Double
Dim ptArr(2) As Double
Dim dirArr(2) As Double

ptArr(0) = 0.0#
ptArr(1) = 0.0#
ptArr(2) = 0.0#
dirArr(0) = 0.0#
dirArr(1) = 0.0#
dirArr(2) = 1.0#
startArr(0) = 1.0#
startArr(1) = 0.0#
startArr(2) = 0.0#

planeSurf = swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr))

Dim profileBody As Body2
Dim extrudedBody As Body2
Dim dirVector As MathVector

profileBody = planeSurf.CreateTrimmedSheet4((swCurve), True)

dirArr(0) = 0.0#
dirArr(1) = 0.0#
dirArr(2) = -1.0#

dirVector = swMath.CreateVector((dirArr))
extrudedBody = swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth)
extrudedBody.Display3(swDocument, RGB(1, 0, 0), 0)

End If
swDocument.ViewZoomtofit()
End Sub

Public swApp As SldWorks

End Class

```

Create Temporary Extruded Body Example (C#)

This example shows how to create a temporary extruded body.

```
//-----
// Preconditions:
// 1. Verify that the specified part document
//    template exists.
// 2. Add a reference to Microsoft.VisualBasic (right-click
//    the name of the project in the Project Explorer, click
//    Add Reference > the .NET tab > Microsoft.VisualBasic >
//    OK.
//
// Postconditions.
// 1. Opens a new part document.
// 2. Creates and selects a sheet (also called a surface) body.
// 3. Creates a temporary extruded body.
// 4. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using Microsoft.VisualBasic;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            SketchManager swSketchManager = default(SketchManager);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            SketchSegment sketchSegment = default(SketchSegment);
            Modeler swModeler = default(Modeler);
            MathUtility swMath = default(MathUtility);
            Body2 profileBody = default(Body2);
            Body2 extrudedBody = default(Body2);
            MathVector dirVector = default(MathVector);
            Surface planeSurf = default(Surface);
            Curve[] trimCurves = new Curve[4];
            double[] points = new double[12];
            object pointArray = null;
            double halfWidth = 0;
            double halfLength = 0;
            double[] startArr = new double[3];
            double[] endArr = new double[3];
            double[] ptArr = new double[3];
            double[] dirArr = new double[3];
            double slotWidth = 0;
            double slotLength = 0;
            double slotDepth = 0;
            bool slotThruAll = false;
            bool status = false;

            swModeler = (Modeler)swApp.GetModeler();
            swMath = (MathUtility)swApp.GetMathUtility();
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2014\\templates\\Part.prtdot", 0, 0, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;

            //Create and select extruded surface body
            points[0] = -0.0720746414289124;
            points[1] = -0.0283600245263074;
            points[2] = 0;
            points[3] = -0.0514715593755;
            points[4] = -0.00345025084396866;
            points[5] = 0;
            points[6] = 0;
            points[7] = 0;
            points[8] = 0;
            points[9] = 0.0872558597840225;
            points[10] = 0.0521037067517796;
            points[11] = 0;
            pointArray = points;
            sketchSegment = (SketchSegment)swSketchManager.CreateSpline((pointArray));
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);
            swFeatureManager.FeatureExtruRefSurface2(true, false, false, 0, 0, 0.0508, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, false, false,
            false);
            swSelectionManager.EnableContourSelection = false;
            status = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);

            slotDepth = 0.01;
            slotWidth = 0.04;
            slotLength = 0.09;
            slotThruAll = false;
            halfWidth = slotWidth / 2;
            halfLength = slotLength / 2;
            ptArr[0] = 0.0;
            ptArr[1] = 0.0;
            ptArr[2] = 0.0;
            dirArr[0] = 0.0;
            dirArr[1] = 0.0;
            dirArr[2] = 1.0;
            startArr[0] = 1.0;
            startArr[1] = 0.0;
            startArr[2] = 0.0;
            planeSurf = (Surface)swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr));

            ptArr[0] = -halfLength;
```

```

ptArr[1] = halfWidth;
ptArr[2] = 0.0;
dirArr[0] = 1.0;
dirArr[1] = 0.0;
dirArr[2] = 0.0;
trimCurves[0] = (Curve)swModeler.CreateLine((ptArr), (dirArr));
trimCurves[0] = (Curve)trimCurves[0].CreateTrimmedCurve2(-halfLength, halfWidth, 0.0, halfLength, halfWidth, 0.0);

ptArr[0] = halfLength;
ptArr[1] = 0.0;
ptArr[2] = 0.0;
startArr[0] = halfLength;
startArr[1] = halfWidth;
startArr[2] = 0.0;
endArr[0] = halfLength;
endArr[1] = -halfWidth;
endArr[2] = 0.0;
dirArr[0] = 0.0;
dirArr[1] = 0.0;
dirArr[2] = -1.0;
trimCurves[1] = (Curve)swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr));
trimCurves[1] = (Curve)trimCurves[1].CreateTrimmedCurve2(halfLength, halfWidth, 0.0, halfLength, -halfWidth, 0.0);

ptArr[0] = halfLength;
ptArr[1] = -halfWidth;
ptArr[2] = 0.0;
dirArr[0] = -1.0;
dirArr[1] = 0.0;
dirArr[2] = 0.0;
trimCurves[2] = (Curve)swModeler.CreateLine((ptArr), (dirArr));
trimCurves[2] = (Curve)trimCurves[2].CreateTrimmedCurve2(halfLength, -halfWidth, 0.0, -halfLength, -halfWidth, 0.0);

ptArr[0] = -halfLength;
ptArr[1] = 0.0;
ptArr[2] = 0.0;
startArr[0] = -halfLength;
startArr[1] = -halfWidth;
startArr[2] = 0.0;
endArr[0] = -halfLength;
endArr[1] = halfWidth;
endArr[2] = 0.0;
dirArr[0] = 0.0;
dirArr[1] = 0.0;
dirArr[2] = -1.0;
trimCurves[3] = (Curve)swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr));
trimCurves[3] = (Curve)trimCurves[3].CreateTrimmedCurve2(-halfLength, -halfWidth, 0.0, -halfLength, halfWidth, 0.0);
profileBody = (Body2)planeSurf.CreateTrimmedSheet((trimCurves));

dirArr[0] = 0.0;
dirArr[1] = 0.0;
dirArr[2] = -1.0;
dirVector = (MathVector)swMath.CreateVector((dirArr));
extrudedBody = (Body2)swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth);
extrudedBody.Display3(swModel, Information.RGB(1, 0, 0), (int)swTempBodySelectOptions_e.swTempBodySelectOptionNone);

```

```

}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

```

```

}
}

```

Create Temporary Extruded Body Example (VBA)

This example shows how to create a temporary extruded body.

```
'-----
' Preconditions: Verify that the specified part document
' template exists.
'
' Postconditions.
' 1. Opens a new part document.
' 2. Creates and selects a sheet (also called a surface) body.
' 3. Creates a temporary extruded body.
' 4. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSketchManager As SldWorks.SketchManager
Dim swSelectionManager As SldWorks.SelectionMgr
Dim sketchSegment As SldWorks.sketchSegment
Dim swModeler As SldWorks.Modeler
Dim swMath As SldWorks.MathUtility
Dim profileBody As SldWorks.Body2
Dim extrudedBody As SldWorks.Body2
Dim dirVector As SldWorks.MathVector
Dim planeSurf As SldWorks.Surface
Dim trimCurves(3) As SldWorks.Curve
Dim points(11) As Double
Dim pointArray As Variant
Dim halfWidth As Double
Dim halfLength As Double
Dim startArr(2) As Double
Dim endArr(2) As Double
Dim ptArr(2) As Double
Dim dirArr(2) As Double
Dim slotWidth As Double
Dim slotLength As Double
Dim slotDepth As Double
Dim slotThruAll As Boolean
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModeler = swApp.GetModeler
    Set swMath = swApp.GetMathUtility

    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swSketchManager = swModel.SketchManager
    Set swModelDocExt = swModel.Extension
    Set swSelectionManager = swModel.SelectionManager

    'Create and select extruded surface body
    points(0) = -7.20746414289124E-02
    points(1) = -2.83600245263074E-02
    points(2) = 0
    points(3) = -0.0514715593755
    points(4) = -3.45025084396866E-03
    points(5) = 0
    points(6) = 0
    points(7) = 0
    points(8) = 0
    points(9) = 8.72558597840225E-02
    points(10) = 5.21037067517796E-02
    points(11) = 0
    pointArray = points
    Set sketchSegment = swSketchManager.CreateSpline((pointArray))
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    swFeatureManager.FeatureExtruRefSurface2 True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, False, False
    swSelectionManager.EnableContourSelection = False
    status = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

    slotDepth = 0.01
    slotWidth = 0.04
    slotLength = 0.09
    slotThruAll = False

    halfWidth = slotWidth / 2
    halfLength = slotLength / 2

    ptArr(0) = 0#
    ptArr(1) = 0#
    ptArr(2) = 0#
    dirArr(0) = 0#
    dirArr(1) = 0#
    dirArr(2) = 1#
    startArr(0) = 1#
    startArr(1) = 0#
```

```

startArr(2) = 0#
Set planeSurf = swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr))

ptArr(0) = -halfLength
ptArr(1) = halfWidth
ptArr(2) = 0#
dirArr(0) = 1#
dirArr(1) = 0#
dirArr(2) = 0#
Set trimCurves(0) = swModeler.CreateLine((ptArr), (dirArr))
Set trimCurves(0) = trimCurves(0).CreateTrimmedCurve2(-halfLength, halfWidth, 0#, halfLength, halfWidth, 0#)

ptArr(0) = halfLength
ptArr(1) = 0#
ptArr(2) = 0#
startArr(0) = halfLength
startArr(1) = halfWidth
startArr(2) = 0#
endArr(0) = halfLength
endArr(1) = -halfWidth
endArr(2) = 0#
dirArr(0) = 0#
dirArr(1) = 0#
dirArr(2) = -1#
Set trimCurves(1) = swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr))
Set trimCurves(1) = trimCurves(1).CreateTrimmedCurve2(halfLength, halfWidth, 0#, halfLength, -halfWidth, 0#)

ptArr(0) = halfLength
ptArr(1) = -halfWidth
ptArr(2) = 0#
dirArr(0) = -1#
dirArr(1) = 0#
dirArr(2) = 0#
Set trimCurves(2) = swModeler.CreateLine((ptArr), (dirArr))
Set trimCurves(2) = trimCurves(2).CreateTrimmedCurve2(halfLength, -halfWidth, 0#, -halfLength, -halfWidth, 0#)

ptArr(0) = -halfLength
ptArr(1) = 0#
ptArr(2) = 0#
startArr(0) = -halfLength
startArr(1) = -halfWidth
startArr(2) = 0#
endArr(0) = -halfLength
endArr(1) = halfWidth
endArr(2) = 0#
dirArr(0) = 0#
dirArr(1) = 0#
dirArr(2) = -1#
Set trimCurves(3) = swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr))
Set trimCurves(3) = trimCurves(3).CreateTrimmedCurve2(-halfLength, -halfWidth, 0#, -halfLength, halfWidth, 0#)

Set profileBody = planeSurf.CreateTrimmedSheet((trimCurves))

dirArr(0) = 0#
dirArr(1) = 0#
dirArr(2) = -1#
Set dirVector = swMath.CreateVector((dirArr))
Set extrudedBody = swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth)
extrudedBody.Display3 swModel, RGB(1, 0, 0), swTempBodySelectOptions_e.swTempBodySelectOptionNone
End Sub

```

Create Temporary Extruded Body Example (VB.NET)

This example shows how to create a temporary extruded body.

```

-----
' Preconditions: Verify that the specified part document
' template exists.
' Postconditions.
' 1. Opens a new part document.
' 2. Creates and selects a sheet (also called a surface) body.
' 3. Creates a temporary extruded body.
' 4. Examine the graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swSketchManager As SketchManager
        Dim swSelectionManager As SelectionMgr
        Dim sketchSegment As SketchSegment
        Dim swModeler As Modeler
        Dim swMath As MathUtility
        Dim profileBody As Body2
        Dim extrudedBody As Body2
        Dim dirVector As MathVector
        Dim planeSurf As Surface
        Dim trimCurves(3) As Curve
        Dim points(11) As Double
        Dim pointArray As Object
        Dim halfWidth As Double
        Dim halfLength As Double
        Dim startArr(2) As Double
        Dim endArr(2) As Double
        Dim ptArr(2) As Double
        Dim dirArr(2) As Double
        Dim slotWidth As Double
        Dim slotLength As Double
        Dim slotDepth As Double
        Dim slotThruAll As Boolean
        Dim status As Boolean

        swModeler = swApp.GetModeler
        swMath = swApp.GetMathUtility
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2014\templates\Part.prt", 0, 0, 0)
        swFeatureManager = swModel.FeatureManager
        swSketchManager = swModel.SketchManager
        swModelDocExt = swModel.Extension
        swSelectionManager = swModel.SelectionManager

        'Create and select extruded surface body
        points(0) = -0.0720746414289124
        points(1) = -0.0283600245263074
        points(2) = 0
        points(3) = -0.0514715593755
        points(4) = -0.00345025084396866
        points(5) = 0
        points(6) = 0
        points(7) = 0
        points(8) = 0
        points(9) = 0.0872558597840225
        points(10) = 0.0521037067517796
        points(11) = 0
        pointArray = points
        sketchSegment = swSketchManager.CreateSpline((pointArray))
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeatureManager.FeatureExtruRefSurface2(True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 0.0174532925199433, 0.017453292519
9433, False, False, False, False, False, False)
        swSelectionManager.EnableContourSelection = False
        status = swModelDocExt.SelectByID("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

        slotDepth = 0.01
        slotWidth = 0.04
        slotLength = 0.09
        slotThruAll = False
        halfWidth = slotWidth / 2
        halfLength = slotLength / 2
        ptArr(0) = 0.0#
        ptArr(1) = 0.0#
        ptArr(2) = 0.0#
        dirArr(0) = 0.0#
        dirArr(1) = 0.0#
        dirArr(2) = 1.0#
        startArr(0) = 1.0#
        startArr(1) = 0.0#
        startArr(2) = 0.0#
        planeSurf = swModeler.CreatePlanarSurface2((ptArr), (dirArr), (startArr))

        ptArr(0) = -halfLength
        ptArr(1) = halfWidth
        ptArr(2) = 0.0#
        dirArr(0) = 1.0#
        dirArr(1) = 0.0#
        dirArr(2) = 0.0#
        trimCurves(0) = swModeler.CreateLine((ptArr), (dirArr))
        trimCurves(0) = trimCurves(0).CreateTrimmedCurve2(-halfLength, halfWidth, 0.0#, halfLength, halfWidth, 0.0#)

        ptArr(0) = halfLength
        ptArr(1) = 0.0#
        ptArr(2) = 0.0#
        startArr(0) = halfLength
    
```

```

startArr(1) = halfWidth
startArr(2) = 0.0#
endArr(0) = halfLength
endArr(1) = -halfWidth
endArr(2) = 0.0#
dirArr(0) = 0.0#
dirArr(1) = 0.0#
dirArr(2) = -1.0#
trimCurves(1) = swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr))
trimCurves(1) = trimCurves(1).CreateTrimmedCurve2(halfLength, halfWidth, 0.0#, halfLength, -halfWidth, 0.0#)

ptArr(0) = halfLength
ptArr(1) = -halfWidth
ptArr(2) = 0.0#
dirArr(0) = -1.0#
dirArr(1) = 0.0#
dirArr(2) = 0.0#
trimCurves(2) = swModeler.CreateLine((ptArr), (dirArr))
trimCurves(2) = trimCurves(2).CreateTrimmedCurve2(halfLength, -halfWidth, 0.0#, -halfLength, -halfWidth, 0.0#)

ptArr(0) = -halfLength
ptArr(1) = 0.0#
ptArr(2) = 0.0#
startArr(0) = -halfLength
startArr(1) = -halfWidth
startArr(2) = 0.0#
endArr(0) = -halfLength
endArr(1) = halfWidth
endArr(2) = 0.0#
dirArr(0) = 0.0#
dirArr(1) = 0.0#
dirArr(2) = -1.0#
trimCurves(3) = swModeler.CreateArc((ptArr), (dirArr), halfWidth, (startArr), (endArr))
trimCurves(3) = trimCurves(3).CreateTrimmedCurve2(-halfLength, -halfWidth, 0.0#, -halfLength, halfWidth, 0.0#)
profileBody = planeSurf.CreateTrimmedSheet((trimCurves))

dirArr(0) = 0.0#
dirArr(1) = 0.0#
dirArr(2) = -1.0#
dirVector = swMath.CreateVector((dirArr))
extrudedBody = swModeler.CreateExtrudedBody(profileBody, dirVector, slotDepth)
extrudedBody.Display3(swModel, RGB(1, 0, 0), swTempBodySelectOptions_e.swTempBodySelectOptionNone)

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Thicken Feature Example (VBA)

This example shows how to create a thicken feature in a multibody part.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates an offset plane and offset surface.
' 2. Selects a face on one body and the offset surface.
' 3. Creates a thicken feature.
' 4. Accesses the thicken feature and gets some property values.
' 5. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swRefPlane As SldWorks.RefPlane
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swPlane As SldWorks.RefPlane
Dim swFeatureThicken As SldWorks.Feature
Dim swFeatureThicken_DEF As SldWorks.ThickenFeatureData
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Create offset surface
    status = swModelDocExt.SelectByID("Top", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swRefPlane = swFeatureMgr.InsertRefPlane(264, 0.01, 0, 0, 0, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "FACE", 2.35573770133328E-02, 0, -2.4476412300487E-03, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", -5.61529049190312E-02, 0, -2.56278005667809E-03, True, 0, Nothing, 0)
    swModel.InsertOffsetSurface 0.01, False

    swModel.ClearSelection2 True

    status = swModelDocExt.SelectByID("Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("Surface-Offset1[1]", "SURFACEBODY", 3.19724221122328E-02, -9.99999999999991E-03, -1.04277742429417E-02, False, 1, Nothing, 0)

    ' Thicken the selected reference surface and then generate a boss
    Set swFeatureThicken = swFeatureMgr.FeatureBossThicken(0.01, 0, 96, False, True, False, True)

    swModel.ClearSelection2 True

    ' Set the property values for the selected feature
    swModel.SelectedFeatureProperties 0, 0, 0, 0, 0, 0, 0, 1, 0, "Thicken1"

    ' Get the thicken feature definition object
    Set swFeatureThicken_DEF = swFeatureThicken.GetDefinition
    swFeatureThicken_DEF.AccessSelections swModel, Nothing

    ' Display whether a boss feature
    Debug.Print "Boss feature? " & swFeatureThicken_DEF.IsBossFeature

    ' Display whether the results of this thicken feature are merged
    Debug.Print "Merged? " & swFeatureThicken_DEF.Merge

    ' Display whether all or only specific bodies were
    ' automatically selected for the thicken feature
    Debug.Print "All bodies automatically selected? " & swFeatureThicken_DEF.AutoSelect

    swFeatureThicken_DEF.ReleaseSelectionAccess

End Sub
```

Create Thin Feature Revolve in Two Directions Example (C#)

This example shows how to create a thin feature revolve in two directions.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\Multiple_Planar_Faces2.sldprt.
//
// Postconditions:
// 1. Creates a thin feature revolve in two directions.
// 2. Examine the graphics area and FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace ThinFeatureRevolve_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            bool boolstatus = false;

            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Axis1", "AXIS", -0.03249248386774, -0.008890295497245, -0.005457395402805,
            true, 16, null, 0);
            boolstatus = Part.Extension.SelectByID2("", "FACE", -0.03948753408952, 0.1016773485926, -0.08343298757264, true, 32,
            null, 0);

            Feature myFeature = default(Feature);
            myFeature = Part.FeatureManager.FeatureRevolve2(false, true, true, false, false, true, 4, 5, 6.28318530718, 0,
            false, true, 0.01, 0.01, 0, 0.002, 0.01, true, true, true);

        }

        public SldWorks swApp;
    }
}
```

Create Thin Feature Revolve in Two Directions Example (VBA)

This example shows how to create a thin feature revolve in two directions.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\Multiple Planar_Faces2.sldprt.  
,
```

```
' Postconditions:  
' 1. Creates a thin feature revolve in two directions.  
' 2. Examine the graphics area and FeatureManager design tree.  
,
```

```
' NOTE: Because the model is used elsewhere, do not save changes  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim Part As SldWorks.ModelDoc2  
    Dim boolstatus As Boolean
```

```
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc
```

```
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)  
    boolstatus = Part.Extension.SelectByID2("Axis1", "AXIS", -0.03249248386774, -0.008890295497245, -0.005457395402805, True, 16,  
Nothing, 0)  
    boolstatus = Part.Extension.SelectByID2("", "FACE", -0.03948753408952, 0.1016773485926, -0.08343298757264, True, 32, Nothing,  
0)
```

```
    Dim myFeature As SldWorks.Feature  
    Set myFeature = Part.FeatureManager.FeatureRevolve2(False, True, True, False, False, True, 4, 5, 6.28318530718, 0, False, True,  
0.01, 0.01, 0, 0.002, 0.01, True, True, True)
```

End Sub

Create Thin Feature Revolve in Two Directions Example (VB.NET)

This example shows how to create a thin feature revolve in two directions.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\Multiple Planar_Faces2.sldprt.  
'-----
```

```
' Postconditions:  
' 1. Creates a thin feature revolve in two directions.  
' 2. Examine the graphics area and FeatureManager design tree.  
'-----
```

```
' NOTE: Because the model is used elsewhere, do not save changes  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim Part As ModelDoc2
```

```
        Dim boolstatus As Boolean
```

```
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Axis1", "AXIS", -0.03249248386774, -0.008890295497245, -0.005457395402805, True,  
16, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.03948753408952, 0.1016773485926, -0.08343298757264, True, 32,  
Nothing, 0)
```

```
        Dim myFeature As Feature
```

```
        myFeature = Part.FeatureManager.FeatureRevolve2(False, True, True, False, False, True, 4, 5, 6.28318530718, 0, False, True,  
0.01, 0.01, 0, 0.002, 0.01, True, True, True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Toolbars Example (C++ COM)

Toolbars are added automatically by the C++ COM Addin Wizard, so you should examine the code that the wizard generates. An example is also included in this topic.

ISldWorks::AddToolBar4 takes bitmap filenames as arguments. You can create those filenames from resources using CreateBitmapFileFromResource().

```
//This method adds the user toolbar to the SOLIDWORKS Design user interface
void CswSampleAddin::AddToolBar()
{
    long docType = swDocTemplateTypePART;
    CComBSTR title;
    CComBSTR callback;
    CComBSTR enable;
    CComBSTR tip;
    CComBSTR hint;
    VARIANT_BOOL added;
    long menuPos = 0; //Not used
    title.LoadString(IDS_TOOLBAR_TITLE);
    CComBSTR smallImageFile, largeImageFile;
    bmp->CreateBitmapFileFromResource(IDB_TOOLBAR_SMALL, &smallImageFile);
    bmp->CreateBitmapFileFromResource(IDB_TOOLBAR_LARGE, &largeImageFile);
    iSwApp->AddToolBar4(addinID, title, smallImageFile, largeImageFile, menuPos, docType, &toolbarID);

    callback.LoadString(IDS_TOOLBAR_CALLBACK0);
    enable.LoadString(IDS_TOOLBAR_ENABLE0);
    tip.LoadString(IDS_TOOLBAR_TIP0);
    hint.LoadString(IDS_TOOLBAR_HINT0);
    iSwApp->AddToolBarCommand2(addinID, toolbarID, 0, callback, enable, tip, hint, &added);
    iSwApp->AddToolBarCommand2(addinID, toolbarID, 1, callback, enable, tip, hint, &added);
    iSwApp->AddToolBarCommand2(addinID, toolbarID, 2, callback, enable, tip, hint, &added);
}
```

Create Triad Manipulator Example (C#)

This example shows how to create a triad manipulator.

```
//-----
// Preconditions:
// 1. Right-click the project name in the Project Explorer and click
//    Add Reference.
// 2. On the Browse tab, navigate to install_dir\api\redist, click
//    SolidWorks.Interop.swpublished.dll, and click OK.
// 3. Right-click the project name in the Project Explorer and click
//    Add > Class.
// 4. Type swDragManipHdlr.cs in Name and click OK.
// 5. Copy Module to SolidWorksMacro.cs.
// 6. Copy Class module to swDragManipHdlr.cs.
// 7. Click Tools > Options and ensure that Stop VSTA debugger on macro exit
//    is not selected.
// 8. Open a model document and select a face.
// 9. Open an Immediate window.
//
// Postconditions:
// 1. Creates a triad manipulator whose origin is the point
//    selected on the face.
// 2. Drag a triad manipulator handle and inspect the Immediate window.
//-----
//Module
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateTriadManipulator_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        swDragManipHdlr swDragHdlr;

        public void Main()
        {
            Manipulator swManip = default(Manipulator);
            TriadManipulator swTriad = default(TriadManipulator);
            Face2 swFace = default(Face2);
            MathUtility swMathUtil = default(MathUtility);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            object vPickPt = null;
            MathPoint swPickPt = default(MathPoint);

            swDragHdlr = new swDragManipHdlr();

            swMathUtil = (MathUtility)swApp.GetMathUtility();
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);

            vPickPt = swSelMgr.GetSelectionPoint2(1, -1);
            swPickPt = (MathPoint)swMathUtil.CreatePoint((vPickPt));
            swModViewMgr = swModel.ModelViewManager;

            swManip = swModViewMgr.CreateManipulator((int)swManipulatorType_e.swTriadManipulator, swDragHdlr);
            swTriad = (TriadManipulator)swManip.GetSpecificManipulator();
            swTriad.Origin = swPickPt;
            swManip.Show(swModel);
        }

        public SldWorks swApp;
    }
}
```

[Back to top](#)

```
//Class module
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using System.Runtime;
```

```

[System.Runtime.InteropServices.ComVisibleAttribute(true)]
public class swDragManipHdlr : SwManipulatorHandler2
{
    public bool OnDelete(object pManipulator)
    {
        Debug.Print("Manipulator deleted");
        return true;
    }

    public void OnDirectionFlipped(object pManipulator)
    {
        Debug.Print("Direction flipped");
    }

    public bool OnDoubleValueChanged(object pManipulator, int handleIndex, ref double Value)
    {
        Debug.Print("Double value changed");
        Debug.Print("  Value = " + Value);
        return true;
    }

    public void OnEndNoDrag(object pManipulator, int handleIndex)
    {
        Debug.Print("Mouse button released");
    }

    public void OnEndDrag(object pManipulator, int handleIndex)
    {
        Debug.Print("Mouse button released after dragging a manipulator handle");
    }

    public void OnHandleRmbSelected(object pManipulator, int handleIndex)
    {
        Debug.Print("Right-mouse button clicked");
        Debug.Print("  HandleIndex = " + handleIndex);
    }

    public void OnHandleSelected(object pManipulator, int handleIndex)
    {
        Debug.Print("Manipulator handle selected");
        Debug.Print("  HandleIndex = " + handleIndex);
    }

    public void OnItemSetFocus(object pManipulator, int Id)
    {
        Debug.Print("Focus set on item");
        Debug.Print("  Item ID = " + Id);
    }

    public bool OnHandleLmbSelected(object pManipulator)
    {
        Debug.Print("Left-mouse button clicked");
        return true;
    }

    public bool OnStringValueChanged(object pManipulator, int handleIndex, ref string Value)
    {
        Debug.Print("String value changed");
        Debug.Print("  String value = " + Value);
        return true;
    }

    public void OnUpdateDrag(object pManipulator, int handleIndex, object newPosMathPt)
    {
        Debug.Print("Manipulator handle moved while left- or right-mouse button depressed");
        Debug.Print("  HandleIndex = " + handleIndex);
    }
}

```

[Back to top](#)

SOLIDWORKS API Help

Create Triad Manipulator Example (VBA)

This example shows how to create a triad manipulator.

```
.....
' Preconditions:
' 1. Click Tools > References > SolidWorks 2015 exposed type libraries for
'   add-in use (swpublished.tlb) and click OK.
' 2. Right-click the project name in the Project Explorer and click
'   Insert > Class Module.
' 3. Click Class1 in the Project Explorer.
' 4. Type swDragManipHdlr in (Name) in the Properties window.
' 5. Copy Module to the main module.
' 6. Copy Class module to the swDragManipHdlr class module.
' 7. Open a model document and select a face.
' 8. Open an Immediate window.
'
' Postconditions:
' 1. Creates a triad manipulator whose origin is the point
'   selected on the face.
' 2. Drag a triad manipulator handle and inspect the Immediate window.
.....
```

Module

```
Option Explicit

Dim swManip          As SldWorks.Manipulator
Dim swTriad          As SldWorks.TriadManipulator
Dim swDragHdlr       As swDragManipHdlr
Dim swFace           As SldWorks.Face2

Sub main()
    Dim swApp          As SldWorks.SldWorks
    Dim swMathUtil     As SldWorks.MathUtility
    Dim swModel        As SldWorks.ModelDoc2
    Dim swModViewMgr   As SldWorks.ModelViewManager
    Dim swSelMgr       As SldWorks.SelectionMgr
    Dim vPickPt        As Variant
    Dim swPickPt       As SldWorks.MathPoint

    Set swDragHdlr = New swDragManipHdlr

    Set swApp = Application.SldWorks
    Set swMathUtil = swApp.GetMathUtility
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)

    vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
    Set swPickPt = swMathUtil.CreatePoint((vPickPt))
    Set swModViewMgr = swModel.ModelViewManager

    Set swManip = swModViewMgr.CreateManipulator(swTriadManipulator, swDragHdlr)
    Set swTriad = swManip.GetSpecificManipulator
    swTriad.Origin = swPickPt
    swManip.Show swModel

End Sub
```

[Back to top](#)

Class module

```
Option Explicit
Implements SwManipulatorHandler2

Private Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean
    Debug.Print "Manipulator deleted"
End Function

Private Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object)
    Debug.Print "Direction flipped"
End Sub
```

```

Private Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As Double) As Boolean
    Debug.Print "Double value changed "
    Debug.Print "   Value           = " & Value
End Function

Private Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "Mouse button released"
    Debug.Print "   HandleIndex as defined in swTriadManipulatorControlPoints_e   = " & handleIndex
End Sub

Private Sub SwManipulatorHandler2_OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "Mouse button released after dragging a manipulator handle"
End Sub

Private Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "Right-mouse button clicked"
    Debug.Print "   HandleIndex as defined in swTriadManipulatorControlPoints_e   = " & handleIndex
End Sub

Private Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "Manipulator handle selected"
    Debug.Print "   HandleIndex           = " & handleIndex
End Sub

Private Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Long)
    Debug.Print "Focus set on item"
    Debug.Print "   Item ID               = " & Id
End Sub

Private Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean
    Debug.Print "Left-mouse button clicked"
End Function

Private Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As String) As Boolean
    Debug.Print "String value changed"
    Debug.Print "   Value                 = " & Value
End Function

Private Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As Object)
    Debug.Print "Manipulator handle moved while left- or right-mouse button depressed"
    Debug.Print "   HandleIndex as defined in swTriadManipulatorControlPoints_e   = " & handleIndex
End Sub

```

[Back to top](#)

Create Triad Manipulator Example (VB.NET)

This example shows how to create a triad manipulator.

```

'-----
' Preconditions:
' 1. Right-click the project name in the Project Explorer and click
'    Add Reference.
' 2. On the Browse tab, navigate to install_dir\api\redist, click
'    SolidWorks.Interop.swpublished.dll, and click OK.
' 3. Right-click the project name in the Project Explorer and click
'    Add > Class.
' 4. Type swDragManipHdlr.vb in Name and click Add.
' 5. Copy Module to SolidWorksMacro.vb.
' 6. Copy Class module to swDragManipHdlr.vb.
' 7. Click Tools > Options and ensure that Stop VSTA debugger on macro exit
'    is not selected.
' 8. Open a model document and select a face.
' 9. Open an Immediate window.
'
' Postconditions:
' 1. Creates a triad manipulator whose origin is the point
'    selected on the face.
' 2. Drag a triad manipulator handle and inspect the Immediate window.
'-----

```

'Module

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

Partial Class SolidWorksMacro

```

    Dim swDragHdlr As swDragManipHdlr

    Sub main()

        Dim swManip As Manipulator
        Dim swTriad As TriadManipulator
        Dim swFace As Face2
        Dim swMathUtil As MathUtility
        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim swSelMgr As SelectionMgr
        Dim vPickPt As Object
        Dim swPickPt As MathPoint

        swDragHdlr = New swDragManipHdlr

        swMathUtil = swApp.GetMathUtility
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFace = swSelMgr.GetSelectedObject6(1, -1)

        vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
        swPickPt = swMathUtil.CreatePoint((vPickPt))
        swModViewMgr = swModel.ModelViewManager

        swManip = swModViewMgr.CreateManipulator(swManipulatorType_e.swTriadManipulator, swDragHdlr)
        swTriad = swManip.GetSpecificManipulator
        swTriad.Origin = swPickPt
        swManip.Show(swModel)

    End Sub

    Public swApp As SldWorks

```

End Class

[Back to top](#)

'Class module

```

Imports SolidWorks.Interop.swpublished
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics

```

```

<System.Runtime.InteropServices.ComVisibleAttribute(True)> _
Public Class swDragManipHdlr
    Implements SwManipulatorHandler2

```

```

    Private Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean Implements SwManipulatorHandler2.OnDelete
        Debug.Print("Manipulator deleted")
    End Function

```

```

    Private Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object) Implements SwManipulatorHandler2.OnDirectionFlipped

```

```

        Debug.Print("Direction flipped")
    End Sub

    Private Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByRef Value As Double) As Boolean
        Debug.Print("Double value changed")
        Debug.Print("  Value = " & Value)
    End Function

    Private Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements SwManipulatorHandler2.OnEndNoDrag
        Debug.Print("Mouse button released")
    End Sub

    Private Sub SwManipulatorHandler2_OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements SwManipulatorHandler2.OnEndDrag
        Debug.Print("Mouse button released after dragging a manipulator handle")
    End Sub

    Private Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements SwManipulatorHandler2.OnHandleRmbSelected
        Debug.Print("Right-mouse button clicked")
        Debug.Print("  HandleIndex = " & handleIndex)
    End Sub

    Private Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements SwManipulatorHandler2.OnHandleSelected
        Debug.Print("Manipulator handle selected")
        Debug.Print("  HandleIndex = " & handleIndex)
    End Sub

    Private Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Integer) Implements SwManipulatorHandler2.OnItemSetFocus
        Debug.Print("Focus set on item")
        Debug.Print("  Item ID = " & Id)
    End Sub

    Private Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean Implements SwManipulatorHandler2.OnHandleLmbSelected
        Debug.Print("Left-mouse button clicked")
    End Function

    Private Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByRef Value As String) As Boolean
        Debug.Print("String value changed")
        Debug.Print("  String value = " & Value)
    End Function

    Private Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByVal newPosMathPt As New PosMathPt) Implements SwManipulatorHandler2.OnUpdateDrag
        Debug.Print("Manipulator handle moved while left- or right-mouse button depressed")
        Debug.Print("  HandleIndex = " & handleIndex)
    End Sub

End Class

```

[Back to top](#)

Create Trimmed Surface Feature Example (C#)

This example shows how to create a trimmed surface feature.

```
// -----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document with two intersecting surface extrude
//    features.
// 2. Selects Surface-Extrude2 as the trim tool and sets the trimming options.
// 3. Trims Surface-Extrude1.
// 4. Creates Surface-Trim1 in the FeatureManager design tree.
// 5. Inspect the Immediate window.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CrateSurfTrimFeatData_CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchMgr = default(SketchManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            SurfaceTrimFeatureData surfTrimFeatData = default(SurfaceTrimFeatureData);
            bool status = false;

            // Create new model document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Create two intersecting surfaces
            status = swModelDocExt.SelectByID2("Right Plane", "Plane", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.068922, 0.023964, 0.0, 0.042733, 0.005543, 0.0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeatureMgr.FeatureExtruRefSurface2(true, false, false, 0, 0, 0.06604, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, false, false,
            false);
            swSelMgr.EnableContourSelection = false;

            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.041529, 0.023059, 0.0, -0.052625, -0.081662, 0.0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeatureMgr.FeatureExtruRefSurface2(false, false, false, 0, 0, 0.0889, 0.06604, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, false, false,
            false);
            swSelMgr.EnableContourSelection = false;

            // Set the trimming options
            status = swFeatureMgr.PreTrimSurface(false, true, false, false);

            // Trim the surface
            status = swModelDocExt.SelectByID2("", "SURFACEBODY", 0.0289416986472588, 0.00781827749557351, 0.0290635845400971, true,
            swFeat = (Feature)swFeatureMgr.PostTrimSurface(true);

            swModel.ClearSelection2(true);

            surfTrimFeatData = (SurfaceTrimFeatureData)swFeat.GetDefinition();

            surfTrimFeatData.AccessSelections(swModel, null);

            Debug.Print(swFeat.Name);
            Debug.Print(" Number of pieces to keep: " + surfTrimFeatData.GetPiecesToKeepCount());
            Debug.Print(" Surface trim feature type as defined in swSurfaceTrimType_e: " + surfTrimFeatData.GetType());
            Debug.Print("");

            object[] varTrimTools = null;
            int i = 0;
```

```

varTrimTools = (object[])surfTrimFeatData.TrimTools;
Debug.Print("Trim tools:");
for (i = 0; i <= surfTrimFeatData.GetTrimToolsCount() - 1; i++)
{
    Debug.Print("  " + ((Feature)varTrimTools[i]).Name);
}

surfTrimFeatData.ReleaseSelectionAccess();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Create Trimmed Surface Feature Example (VBA)

This example shows how to create a trimmed surface feature.

```
' -----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with two intersecting surface extrude
'    features.
' 2. Selects Surface-Extrude2 as the trim tool and sets the trimming options.
' 3. Trims Surface-Extrude1.
' 4. Creates Surface-Trim1 in the FeatureManager design tree.
' 5. Inspect the Immediate window.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchMgr As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureMgr As SldWorks.FeatureManager
Dim surfTrimFeatData As SldWorks.SurfaceTrimFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim status As Boolean

Sub main()
    Set swApp = Application.SldWorks

    ' Create new model document
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set swSketchMgr = swModel.SketchManager
    Set swModelDocExt = swModel.Extension
    Set swFeatureMgr = swModel.FeatureManager
    Set swSelMgr = swModel.SelectionManager

    ' Create two intersecting surfaces
    status = swModelDocExt.SelectByID("Right Plane", "Plane", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateLine(-0.068922, 0.023964, 0#, 0.042733, 0.005543, 0#)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureMgr.FeatureExtruRefSurface2 True, False, False, 0, 0, 0.06604, 0.00254, False, False, False, False, 1.74532925199433E-
02, 1.74532925199433E-02, False, False, False, False, False, False, False, False
    swSelMgr.EnableContourSelection = False

    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateLine(-0.041529, 0.023059, 0#, -0.052625, -0.081662, 0#)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureMgr.FeatureExtruRefSurface2 False, False, False, 0, 0, 0.0889, 0.06604, False, False, False, False, 1.74532925199433E-
02, 1.74532925199433E-02, False, False, False, False, False, False, False, False
    swSelMgr.EnableContourSelection = False

    ' Set the trimming options
    status = swFeatureMgr.PreTrimSurface(False, True, False, False)

    ' Trim the surface
    status = swModelDocExt.SelectByID("", "SURFACEBODY", 2.89416986472588E-02, 7.81827749557351E-03, 2.90635845400971E-02, True,
0, Nothing, 0)
    Set swFeat = swFeatureMgr.PostTrimSurface(True)

    swModel.ClearSelection2 True

    Set surfTrimFeatData = swFeat.GetDefinition

    surfTrimFeatData.AccessSelections swModel, Nothing

    Debug.Print swFeat.Name
    Debug.Print " Number of pieces to keep: " & surfTrimFeatData.GetPiecesToKeepCount
    Debug.Print " Surface trim feature type as defined in swSurfaceTrimType_e: " & surfTrimFeatData.GetType
    Debug.Print ""

    Dim varTrimTools As Variant
    Dim i As Long

    varTrimTools = surfTrimFeatData.TrimTools
    Debug.Print "Trim tools:"
```

```
For i = 0 To surfTrimFeatData.GetTrimToolsCount - 1
    Debug.Print " " & varTrimTools(i).Name
Next

surfTrimFeatData.ReleaseSelectionAccess

End Sub
```

Create Trimmed Surface Feature Example (VB.NET)

This example shows how to create a trimmed surface feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with two intersecting surface extrude
'   features.
' 2. Selects Surface-Extrude2 as the trim tool and sets the trimming options.
' 3. Trims Surface-Extrude1.
' 4. Creates Surface-Trim1 in the FeatureManager design tree.
' 5. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swSketchMgr As SketchManager
    Dim swModelDocExt As ModelDocExtension
    Dim swSketchSegment As SketchSegment
    Dim swFeatureMgr As FeatureManager
    Dim surfTrimFeatData As SurfaceTrimFeatureData
    Dim swSelMgr As SelectionMgr
    Dim swFeat As Feature
    Dim status As Boolean

```

Sub main()

```

    ' Create new model document
    swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swSketchMgr = swModel.SketchManager
    swModelDocExt = swModel.Extension
    swFeatureMgr = swModel.FeatureManager
    swSelMgr = swModel.SelectionManager

    ' Create two intersecting surfaces
    status = swModelDocExt.SelectByID("Right Plane", "Plane", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch(True)
    swSketchSegment = swSketchMgr.CreateLine(-0.068922, 0.023964, 0.0#, 0.042733, 0.005543, 0.0#)
    swModel.ClearSelection2(True)
    status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureMgr.FeatureExtruRefSurface2(True, False, False, 0, 0, 0.06604, 0.00254, False, False, False, False, 0.017453292519)
    swSelMgr.EnableContourSelection = False

    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchMgr.InsertSketch(True)
    swSketchSegment = swSketchMgr.CreateLine(-0.041529, 0.023059, 0.0#, -0.052625, -0.081662, 0.0#)
    swModel.ClearSelection2(True)
    status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureMgr.FeatureExtruRefSurface2(False, False, False, 0, 0, 0.0889, 0.06604, False, False, False, False, 0.017453292519)
    swSelMgr.EnableContourSelection = False

    ' Set the trimming options
    status = swFeatureMgr.PreTrimSurface(False, True, False, False)

    ' Trim the surface
    status = swModelDocExt.SelectByID("", "SURFACEBODY", 0.0289416986472588, 0.00781827749557351, 0.0290635845400971, True, 0,
    swFeat = swFeatureMgr.PostTrimSurface(True)

    swModel.ClearSelection2(True)

    surfTrimFeatData = swFeat.GetDefinition

    surfTrimFeatData.AccessSelections(swModel, Nothing)

    Debug.Print(swFeat.Name)
    Debug.Print(" Number of pieces to keep: " & surfTrimFeatData.GetPiecesToKeepCount)
    Debug.Print(" Surface trim feature type as defined in swSurfaceTrimType_e: " & surfTrimFeatData.GetType)
    Debug.Print("")

    Dim varTrimTools As Object
    Dim i As Integer

    varTrimTools = surfTrimFeatData.TrimTools
    Debug.Print("Trim tools:")
    For i = 0 To surfTrimFeatData.GetTrimToolsCount - 1
        Debug.Print(" " & varTrimTools(i).Name)

```

Next

surfTrimFeatData.**ReleaseSelectionAccess**()

End Sub

Public swApp As SldWorks

End Class

Create Unfolded View Example (C#)

This example shows how to create an unfolded view from an existing view.

```
//-----
// Preconditions: Open:
//   public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw
//
// Postconditions: A new unfolded view is created from Drawing View1.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertUnfoldedView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc ddoc;
        View myView;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            ddoc = (DrawingDoc)Part;
            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            myView = ddoc.CreateUnfoldedViewAt3(0.379074752406062, 0.276482735105582, 0, false);
        }

        public SldWorks swApp;
    }
}
```

Create Unfolded View Example (VBA)

This example shows how to create an unfolded view from an existing view.

```
'-----  
' Preconditions: Open:  
'   public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw  
'  
' Postconditions: A new unfolded view is created from Drawing View1.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim myView As SldWorks.View  
Dim boolstatus As Boolean  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
    Set myView = Part.CreateUnfoldedViewAt3(0.379074752406062, 0.276482735105582, 0, False)  
  
End Sub
```

Create Unfolded View Example (VB.NET)

This example shows how to create an unfolded view from an existing view.

```
'-----  
' Preconditions: Open:  
'   public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw  
'-----  
' Postconditions: A new unfolded view is created from Drawing View1.  
'-----  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2  
    Dim myView As View  
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc  
        boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
        myView = Part.CreateUnfoldedViewAt3(0.379074752406062, 0.276482735105582, 0, False)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Variable-pitch Helix Example (C#)

This example shows how to create a variable-pitch helix.

```
//-----
// Preconditions: Verify that the specified part document
// template exists.
//
// Postconditions:
// 1. Creates a variable-pitch helix.
// 2. Examine the graphics area and FeatureManager design
//    tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace AddVariablePitchHelixCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            int errors = 0;
            bool status = false;

            // Create part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            // Sketch a circle
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.045549, 0.013926, 0.0);

            // Create a variable-pitch helix using the sketched circle
            status = swFeatureManager.InsertVariablePitchHelix(false, true, 1, 4.712388980385);
            status = swFeatureManager.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268);
            status = swFeatureManager.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.5);
            status = swFeatureManager.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.75);
            status = swFeatureManager.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875);
            status = swFeatureManager.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1);
            swFeature = (Feature)swFeatureManager.EndVariablePitchHelix();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Create Variable-pitch Helix Example (VBA)

This example shows how to create a variable-pitch helix.

```
'-----  
' Preconditions: Verify that the specified part document  
' template exists.  
'  
' Postconditions:  
' 1. Creates a variable-pitch helix.  
' 2. Examine the graphics area and FeatureManager design tree.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSketchManager As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swFeature As SldWorks.Feature  
Dim errors As Long  
Dim status As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    ' Create part document  
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)  
    Set swModel = swApp.ActiveDoc  
    Set swSketchManager = swModel.SketchManager  
    Set swFeatureManager = swModel.FeatureManager  
  
    ' Sketch a circle  
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.045549, 0.013926, 0#)  
  
    ' Create a variable-pitch helix using the sketched circle  
    status = swFeatureManager.InsertVariablePitchHelix(False, True, 1, 4.712388980385)  
    status = swFeatureManager.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268)  
    status = swFeatureManager.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.5)  
    status = swFeatureManager.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.75)  
    status = swFeatureManager.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875)  
    status = swFeatureManager.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1)  
    Set swFeature = swFeatureManager.EndVariablePitchHelix()  
  
End Sub
```

Create Variable-pitch Helix Example (VB.NET)

This example shows how to create a variable-pitch helix.

```
'-----
' Preconditions: Verify that the specified part document
' template exists.
'
' Postconditions:
' 1. Creates a variable-pitch helix.
' 2. Examine the graphics area and FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim errors As Integer
        Dim status As Boolean

        ' Create part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swModel = swApp.ActiveDoc
        swSketchManager = swModel.SketchManager
        swFeatureManager = swModel.FeatureManager

        ' Sketch a circle
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.045549, 0.013926, 0.0#)

        ' Create a variable-pitch helix using the sketched circle
        status = swFeatureManager.InsertVariablePitchHelix(False, True, 1, 4.712388980385)
        status = swFeatureManager.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268)
        status = swFeatureManager.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.5)
        status = swFeatureManager.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.75)
        status = swFeatureManager.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875)
        status = swFeatureManager.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1)
        swFeature = swFeatureManager.EndVariablePitchHelix()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```